



Leopold–Franzens–University
Innsbruck

Institute of Computer Science
Distributed and Parallel Systems

Adaptive Resource Scaling for Data Stream Processing Clusters

Bachelor Thesis

Supervisors: Juan Aznar Poveda, Stefan Pedratscher

Emanuel Prader
12116213

Innsbruck, Austria
2 October 2026

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt durch meine eigenhändige Unterschrift, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder inhaltlich den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht. Ich erkläre mich mit der Archivierung der vorliegenden Bachelorarbeit einverstanden.

02.10.2026

Datum



Unterschrift

Unterzeichner	Emanuel Maximilian Prader
Datum und Uhrzeit	02.10.2026, 16:20 (GMT+02:00)
	ID Austria
Dieses Dokument ist digital signiert	
Dieses mit einer qualifizierten elektronischen Signatur versehene Dokument hat gemäß Art. 25 Abs. 2 der Verordnung (EU) Nr. 910/2014 vom 23. Juli 2014 ("EIDAS-VO") die gleiche Rechtswirkung wie ein handschriftlich unterschriebenes Dokument.	
Informationen zur Prüfung der elektronischen Signatur finden Sie unter: https://www.signaturpruefung.gv.at	

Acknowledgements

During the preparation of this work, the author used Claude Sonnet 5 as well as Gemini 3.6 Flash in order to improve the structure, readability, and language of the manuscript. After using those services, the author reviewed and edited the content as needed and takes full responsibility for the content and functionality of the submitted work, including the code base. Results presented in this thesis were obtained using the Chameleon testbed supported by the National Science Foundation.

Abstract

Distributed Stream Processing (DSP) systems must continuously match resource allocation to workloads whose data rates fluctuate unpredictably: static provisioning either wastes capacity during quiet periods or risks growing backlogs and violating service-level objectives during bursts. Autoscaling mechanisms such as the Kubernetes Horizontal Pod Autoscaler (HPA) rely on system-level metrics like CPU and memory utilization, which are only indirect indicators of whether a streaming pipeline is keeping up with its input, while existing predictive approaches are often limited to a narrow set of generic metrics, evaluated on static or non-containerized infrastructure, or built on computationally expensive, non-incremental forecasting models unsuited to short reaction timescales. This thesis presents UpStream, a hybrid reactive/predictive autoscaler for Apache Flink on Kubernetes that treats the DSP engine as a black box and governs the size of its underlying container pool using domain-specific engine metrics rather than generic utilization figures. UpStream combines two coordinated control loops: an internal loop, native to the DSP engine, that adapts operator parallelism to the currently available task slots, and an external loop that pairs a reactive gate responding immediately to backpressure with a Model Predictive Control (MPC) component that forecasts busy and idle time over a short horizon and computes the container pool size that drives both toward their target values. A greedy allocation step then maps the resulting task-slot demand onto compute nodes of varying sizes, favoring fewer, larger containers to reduce network and orchestration overhead. UpStream is evaluated against a CPU- and memory-tuned HPA baseline on a bare-metal Chameleon Cloud testbed, using pipelines of RIOTBench ported to Apache Flink through Apache Beam, under constant, exponential-ramp, and realistic TAXI-trace load profiles. On the backpressure-bound ETL pipeline, UpStream increases output throughput by a factor of 2.1 to 5.7 over HPA while reducing the time spent above 10% backpressure from 86 – 89% to 36 – 47%. On the lighter STATS pipeline, where CPU and memory already correlate well with load, UpStream improves its target metrics but yields comparable or lower output throughput, in one profile matching HPA’s insert rate while provisioning 38% fewer task slots. On the PRED pipeline, backpressure remains negligible throughout, so UpStream stays at its minimal configuration while HPA provisions roughly twice as many task slots based on utilization alone. These results indicate that DSP-specific signals provide a substantial advantage over generic infrastructure metrics precisely when a pipeline is bottlenecked in ways CPU and memory utilization do not reveal but offer limited additional benefit and a slower reaction time on pipelines whose load is already visible to generic metrics.

Contents

Acknowledgements	iii
1 Introduction	1
2 Related Work	5
3 UpStream	8
3.1 Abstract Application Model	8
3.2 Execution Model and Runtime Topology	9
3.3 System Architecture Overview	10
3.3.1 Layered Scaling Model	11
3.3.2 (i) Internal Scaling Loop (DSP Engine)	12
3.3.3 (ii) External Reactive / Predictive Scaling Loop (UpStream)	12
3.4 Control Loop at a Glance	13
3.5 Handling Backpressure	14
3.5.1 Proportional Scaling Factor	15
3.6 Model Predictive Control for Resource Scaling	16
3.6.1 System Dynamics Model	18
3.6.2 Implemented Cost Function	19
3.7 Compute Node Allocation	21
4 Stream Processing Applications	24
4.1 Application Architecture	24
4.2 Pipelines	25
4.2.1 ETL	25
4.2.2 STATS	25
4.2.3 TRAIN	26
4.2.4 PRED	26
4.3 Summary of the Application Suite	27
5 Datasets and Workloads	30
5.1 Datasets	31
5.1.1 TAXI	31
5.1.2 FIT	31
5.1.3 GRID	31

5.2	Load Profiles	31
5.2.1	Load Profile Generation	32
5.2.2	Constant Load	32
5.2.3	Exponential Ramp	33
5.2.4	Realistic TAXI Profile	33
5.3	Event Generator	35
6	Evaluation and Results	36
6.1	Infrastructure Setup	36
6.2	Application Deployment	37
6.3	Scaler Configuration	39
6.4	Hyperparameters	39
6.5	Results	42
6.6	ETL Pipeline Evaluation	42
6.6.1	Throughput and Ingestion Performance	42
6.6.2	Backpressure, Resource Provisioning, and Execution Dynamics	43
6.6.3	Key Takeaways	45
6.7	STATS Pipeline Evaluation	45
6.7.1	Throughput and Output Performance	45
6.7.2	Target Metric Attainment	47
6.7.3	Resource Provisioning and Utilization	48
6.7.4	Why Improved Target Metrics Do Not Translate into Better Performance	48
6.7.5	Key Takeaways	50
6.8	PRED Pipeline Evaluation	53
6.8.1	Capacity Based on Backpressure	53
6.8.2	Resource Provisioning	54
6.8.3	Resource utilization and Short-Term Resource Spikes	54
6.8.4	Scaling Dynamics	57
6.8.5	Busy and Idle Time	57
6.8.6	Limitations	59
6.8.7	Key Takeaways	59
7	Conclusion	60
7.1	Summary	60
7.2	Discussion	61
7.3	Limitations	62
7.4	Future Work	64
	Bibliography	66

1 Introduction

Stream processing is a computational paradigm that enables the continuous ingestion, processing, and transformation of data as it is generated [1, 29]. Unlike batch processing [8, 41], which processes data in discrete chunks, stream processing systems analyze data in motion, allowing for low-latency insights and real-time decision-making [13, 7, 12]. These systems consist of operators that transform incoming data streams, often running in parallel across multiple processors to handle high data rates efficiently [2, 5, 29, 12]. The primary advantages of stream processing include reduced processing latency, the ability to handle high-throughput data streams, and improved responsiveness in dynamic environments [6, 31, 2, 37, 29, 1]. Stream processing is widely used in applications requiring real-time data analysis. For example, in the finance sector, banks leverage stream processing for real-time fraud detection and high-frequency trading by monitoring transaction streams [36]. In healthcare, continuous patient monitoring and real-time data analysis from medical devices assist in early diagnosis and workload reduction for doctors [3]. Large amounts of medical data, such as DNA, are processed in real time via stream processing approaches [26]. Smart cities utilize stream processing for real-time traffic management, public transportation optimization, and environmental monitoring (e.g., air quality or weather) to enhance urban planning [35, 40, 29, 1].

Despite these benefits, Distributed Stream Processing (DSP) systems [6, 12] face a fundamental operational challenge: workloads in real-world streaming applications are rarely stable [11, 12]. Data rates fluctuate with time of day, seasonal patterns, or unpredictable bursts. This could be a sudden spike in transactions during a flash sale or a surge in sensor readings during an extreme weather event. The resources required to process them fluctuate accordingly. Provisioning a DSP system for peak load wastes resources during quieter periods, while provi-

sioning for average load risks growing backlogs, increased latency, and violated service-level objectives when demand spikes [31, 11]. Manually resizing a cluster in response to these shifts is impractical at the timescales stream processing operates on, motivating the need for autoscaling: Autoscaling describes mechanisms that automatically adjust the resources allocated to a running DSP application in response to changing workload conditions, ideally without human intervention and with minimal disruption to the running job [31, 7]. Autoscaling methods in distributed systems typically rely on system-level metrics, such as CPU and memory utilization, to dynamically adjust resource allocation based on workload demands. While these metrics are broadly applicable, DSP applications provide more involved metrics such as average busy time or backpressure [13, 6, 7, 12].

The default autoscaling mechanism in Kubernetes, Horizontal Pod Autoscaling (HPA)¹ is reactive, adjusting resources only after demand has already increased, which can result in temporary resource shortages or over-provisioning [20]. To address this, predictive approaches have been proposed that forecast workload changes ahead of time [6], ranging from hidden Markov models [33] and ARIMA-based forecasting [32, 19] to reinforcement learning [25] and dedicated predictive horizontal scalers for Apache Flink on Kubernetes [30, 38]. However, these approaches are frequently constrained by a narrow focus on generic metrics such as CPU utilization [19], evaluation on static or non-containerized infrastructure [33], or computationally expensive, non-incremental prediction models that limit responsiveness at short timescales [32].

This thesis addresses these limitations by proposing UpStream, a Model Predictive Control (MPC)-based prediction and scaling approach that leverages multiple domain-specific DSP metrics within a containerized Apache Flink² [5] environment, aiming to combine the responsiveness of predictive scaling with the operational realism of production-representative workloads. The proposed autoscaler is therefore a hybrid reactive/predictive framework structured around two coordinated control loops. An internal loop, managed natively by a DSP engine like Apache Flink, adapts operator parallelism to fully exploit whatever resources are currently available. An external loop, implemented by UpStream, treats the DSP

¹<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>

²<https://flink.apache.org/>

engine as a black box and instead governs the size of the underlying container pool. This external loop combines two modes of operation: a reactive gate that responds immediately to backpressure, an asymmetric metric that only indicates underprovisioning, and, when the system is not backpressured, an MPC component (detailed in Section 3.6) that forecasts busy and idle time over a short horizon and computes the resource allocation that best drives both metrics toward their target values. Once a target number of task slots has been determined, a separate allocation step maps them onto concretely sized containers, favoring fewer, larger containers where possible to reduce the network and management overhead associated with purely horizontal scaling.

By incorporating workload-specific indicators such as busy time, idle time, and backpressure, this approach contrasts with traditional autoscaling mechanisms that rely solely on system-level metrics like CPU and memory usage. Structuring the controller around a reactive gate and a predictive optimization loop further distinguishes UpStream from prior predictive approaches, which typically apply a single forecasting model uniformly regardless of how close the system already is to a critical state, thereby risking either a sluggish response to sudden bursts or unnecessary oscillation under normal conditions. The objective is to explore whether the integration of these domain-specific metrics and this layered control structure leads to more tailored, workload-aware scaling strategies, thereby enhancing the predictive power and stability of scaling mechanisms. Such an approach makes the scaler more sensitive to the unique characteristics of DSP workloads, ultimately aiming to improve the overall efficiency and performance of the system while reducing the operational overhead of managing it.

The primary contributions of this thesis are as follows:

- **UpStream Autoscaler Design:** A hybrid, dual-loop autoscaling framework for containerized stream processing that combines native task parallelism with external container pool management.
- **Incremental MPC Control:** A predictive scaling mechanism based on Model Predictive Control (MPC) that utilizes domain-specific metrics (busy time, idle time, and backpressure) to forecast control actions and prevent resource oscillation.
- **Resource Optimization:** An integrated container-allocation strategy that

optimizes slot-to-container mapping, preferring larger container instances to minimize network and orchestration overhead.

- **Empirical Evaluation:** A comprehensive experimental evaluation on the Chameleon Cloud [24] testbed comparing UpStream against a standard Kubernetes HPA baseline under realistic streaming workloads.

The remainder of this thesis is structured as follows. Chapter 2 reviews related work on autoscaling and resource elasticity for distributed stream processing systems, positioning UpStream relative to existing reactive, predictive, and MPC-based approaches. Chapter 3 introduces UpStream in detail, presenting the abstract application model, the execution model and runtime topology, the layered dual-loop scaling architecture, the backpressure-driven reactive gate, the MPC-based predictive controller, and the compute node allocation strategy. Chapter 4 describes the stream processing applications used to evaluate UpStream, including the application architecture and the ETL, STATS, TRAIN, and PRED pipelines. Chapter 5 presents the datasets and workloads, covering the datasets, the generation of constant, exponential-ramp, and realistic load profiles, and the event generator used to replay them. Chapter 6.5 reports the evaluation and results, describing the infrastructure setup, application deployment, scaler configuration, and hyperparameters, followed by a pipeline-by-pipeline analysis of throughput, resource provisioning, and scaling dynamics for ETL, STATS, and PRED. Finally, Chapter 7 concludes the thesis with a summary of the findings, a discussion of the conditions under which UpStream is beneficial, a statement of the evaluation’s limitations, and directions for future work.

2 Related Work

In the domain of scaling Data Stream Processing (DSP) jobs, numerous studies have addressed various aspects of scalability and resource elasticity. This chapter discusses related work on scaling DSP jobs, covering job-level scaling of operator parallelism and resource-level elasticity of the underlying compute infrastructure, and identifies where this thesis differs.

Kalavri et al. [22] developed a mechanism-agnostic scaling controller for DSP jobs called DS2. This controller analyses the true output of each node in the job graph and considers the data flow topology to assess the resulting load on successor nodes. However, DS2 relies on specific metrics not natively available in the Apache Flink runtime, necessitating the implementation of a custom MetricsManager. By leveraging native Apache Flink metrics, one could potentially eliminate this overhead. While DS2 effectively scales the parallelism of an Apache Flink job, it does not extend to scaling the Apache Flink cluster itself in response to increased demand.

Liu et al. [28] introduced a metric-based performance model for scaling operator parallelism in Apache Flink jobs. Their model focuses on CPU utilization and thread clock time, categorizing operator performance based on CPU competition into three states: non-, non-full-, and full-competition. Although this model slightly outperforms DS2, its limited pool of metrics may restrict its overall effectiveness. Future research could benefit from exploring additional metrics, such as backpressure.

When considering the resource elasticity of DSP systems, effective monitoring, analysis, and scaling policy development are critical [7]. Existing approaches, such as those by Runsewe et al. [33], Kumbhare et al. [27], and Imai et al. [16], primarily focus on deploying DSP systems on virtual machines (VMs). While this approach is feasible, it complicates horizontal scalability compared to container orchestration platforms like Kubernetes.

Runsewe et al. [33] proposed a multidimensional hidden Markov model to predict resource requirements for Apache Spark jobs. Their experiments were conducted on a static Spark cluster, meaning they did not assess the actual performance of scaling infrastructure or its capacity to adapt swiftly to changes in resource needs.

Khaleq et al. [25] introduced a reinforcement learning agent that selects the scaling metric (CPU or memory) for the Kubernetes Horizontal Pod Autoscaler in order to minimize response time. They evaluated it on standalone Python web app microservices deployed as pods on Google Kubernetes Engine and compared it against the default HPA in terms of response time. However, their benchmarking lacks integration with a DSP system, as the pods were stateless request/response services rather than operators of a continuously running stream processing job (e.g., in Apache Flink), so effects such as backpressure and operator dependencies were not considered.

Pfister et al. [30] presented Daedalus, a predictive horizontal autoscaling model concentrating on CPU utilization and throughput in an Apache Flink cluster deployed on Kubernetes. Each Apache Flink job is run in application mode, allowing for resource isolation. In addition to a horizontal scaling approach, this thesis attempts to improve the performance of the Apache Flink cluster by bundling task managers into larger instances as scaling progresses to reduce the networking communication overhead between task managers.

Varga et al. [38] proposed a scaling architecture for Apache Flink clusters that combines the Kubernetes HPA with an Apache Flink Kubernetes operator. Their policy uses two metrics: the relative change rate of the Kafka consumer lag and task utilization derived from idle time. However, they do not evaluate the policy under dynamic load. Their experiments use a synthetic job with a single stateful operator and manually triggered scaling operations and only measure the downtime of scaling (savepointing and state restoration). Consequently, the scaling behavior under diverse load patterns and the resulting resource utilization remain unexamined.

Incerto et al. [17] explored the application of a model predictive controller (MPC) using a multi-class queuing network (QN) to optimize horizontal and vertical scaling in a virtualized compute cluster. Their method dynamically adjusts load balancing and resource-sharing policies, achieving improved performance metrics,

including response time and throughput in heterogeneous environments. This thesis looks into the specific impact of MPC for DSP workflows.

Roy et al. [32] developed an MPC-based resource allocation algorithm for cloud environments such as Amazon EC2, which provisions resources by anticipating workload fluctuations and balancing the risks of under- and over-provisioning. Their workload forecasting relies on ARIMA models, which are computationally intensive and non-incremental. In contrast, this thesis uses an MPC controller whose internal model captures the dynamics of the DSP system, so scaling decisions are derived directly from predicted system behavior without a separate forecasting stage.

Additionally, Joshi et al. [19] employed an ARIMA model but opted for a feedback-based Proportional Integral Derivative (PID) controller as described by Mark Willis [39]. Their ARIMA-PID approach notably reduced system overutilization compared to the native Horizontal Pod Autoscaler of Kubernetes. However, their focus was limited to CPU utilization and HPA, whereas this thesis encompasses multiple domain-specific metrics in the context of a more complex stream processing application.

Overall, the existing literature provides a robust foundation for understanding DSP job scaling and resource elasticity. This thesis builds upon these insights by proposing an alternative approach that integrates incremental prediction and resource allocation, aimed at optimizing both performance and efficiency in containerized DSP environments.

3 UpStream

This chapter introduces **UpStream**, a hybrid reactive/predictive auto-scaling framework designed for stream processing systems. UpStream aims to improve upon reactive scaling by leveraging domain-specific metrics from the processing engine to inform its scaling decisions. At its core, UpStream employs a Model Predictive Control (MPC) approach to predict control behavior and optimize resource allocation accordingly. The method is designed to be adaptable across stream processing platforms and scalable with varying workloads. The following sections describe the architecture, its integration with DSP engines, and the underlying MPC-based control strategy.

3.1 Abstract Application Model

At a high level of abstraction, data stream processing (DSP) applications are commonly modelled as directed graphs that define the flow and transformation of data through a pipeline of processing components. This abstract application model comprises *vertices* representing data sources, operators, and data sinks, as well as *edges*, which model the continuous data streams flowing between them. Formally, the application is described as a directed graph $G_{dsp} = (V, E)$, where $V = V_{src} \cup V_{op} \cup V_{sink}$ denotes the set of vertices and E the set of edges connecting them. Source vertices V_{src} emit input data and have no incoming edges, as these are considered the source vertices, where data is received from external sources such as Apache Kafka, while sink vertices V_{sink} consume results by forwarding them to some external entity or potentially even dropping all data received. They have no outgoing edges. Operators V_{op} apply user-defined or built-in functions to incoming data units, enabling stateless transformations (e.g., filtering) or stateful computations (e.g., windowed aggregation or joins). Apart from the conventional DAG structure for DSP applications, modern frameworks,

such as Apache Flink, increasingly support cyclic topologies to facilitate iterative processing workflows. Moreover, data streams can be broadcast to multiple downstream operators, effectively modelling hyperedges in the graph to support parallel, multi-query execution. This abstract representation serves as a foundational layer for understanding and optimizing DSP applications before mapping them to a concrete execution environment [21, 14, 2].

3.2 Execution Model and Runtime Topology

While the abstract model defines the logical structure of a DSP application, the execution model implements this specification by introducing implementation-level details that guide deployment on a physical system. The resulting execution graph $\bar{G}_{dsp}$ replaces high-level components with concrete entities such as connectors to external systems (for sources and sinks) and processing threads or tasks (for operators). Similar to the abstract model, edges in this graph represent the flow of data between components but are now mapped to communication layers such as network sockets or multi-thread communication mechanisms such as shared memory [14].

To fully utilize modern distributed systems, DSP frameworks exploit multiple forms of parallelism. *Task parallelism* arises naturally from the graph structure, allowing independent operators and processing paths to execute concurrently [14, 2]. *Pipeline parallelism* means operators in the same processing path handle different elements of the stream simultaneously (e.g., while one operator processes tuple i , its successor processes tuple $i - 1$) [14, 2]. Most critically, *data parallelism* enables parallel processing by instantiating multiple replicas of a task, each consuming a partition of the input stream. For each operator $v \in V_{op}$, the execution model introduces $n \geq 1$ parallel tasks $\bar{v}_1, \dots, \bar{v}_n$ that operate independently [14, 2].

A key optimization performed during the derivation of the execution graph is *operator fusion*, also referred to as *operator chaining*. This process combines adjacent operators from the application DAG into a single executable task, reducing serialization, communication overhead, and context-switching costs. From the perspective of the application DAG, this means that multiple logical operators, for example, u followed by v are merged into a single physical task x in the exe-

cution graph, such that $f_x(t) = f_v(f_u(t))$. Fusion is typically applied to operators with compatible resource demands and stateless behavior. Modern frameworks may extend fusion support to certain stateful scenarios. This transformation maintains the semantics of the original graph while improving performance and resource efficiency [14].

The mapping of execution tasks to physical resources is determined by the system’s *scheduler*. The scheduler is responsible for placing operator instances onto the available computational slots within a cluster, often balancing workload across nodes while respecting constraints such as data locality, operator affinities, and fault tolerance. Static schedulers rely on pre-defined deployment plans, whereas dynamic or elastic schedulers, such as those required for auto-scaling systems, continuously adapt placements based on runtime metrics. The runtime behavior of such scheduling mechanisms is central to UpStream’s ability to dynamically adjust resource usage while maintaining performance and consistency [14, 2].

Another key aspect of modern DSP systems is their ability to adapt the parallelism of the execution graph $\tilde{G}_{dsp}$ at runtime. The system can dynamically replace an operator v with a new set of parallel instances $\{\bar{v}'_1, \dots, \bar{v}'_{n'}\}$, where $n' \neq n$, and redistribute incoming stream partitions accordingly. For stateful operators, the system ensures correct redistribution and restoration of operator state using checkpointing or savepoint mechanisms. These adaptations are orchestrated by a dedicated runtime component (e.g. rescaling controller), which coordinates container-level changes and operator redeployment without interrupting overall job execution [14].

3.3 System Architecture Overview

UpStream is designed as a layered auto-scaling framework that orchestrates elasticity in Data Stream Processing (DSP) systems by combining internal rescaling with external infrastructure scaling. The system comprises two control loops: an *internal rescaling loop* managed by the DSP engine annotated as (i_x) steps in the system architecture diagram 3.1 and an *external scaling loop* driven by UpStream annotated as (ii_x) . This architecture enables the system to respond reactively to current load conditions while proactively preparing for future workload trends.

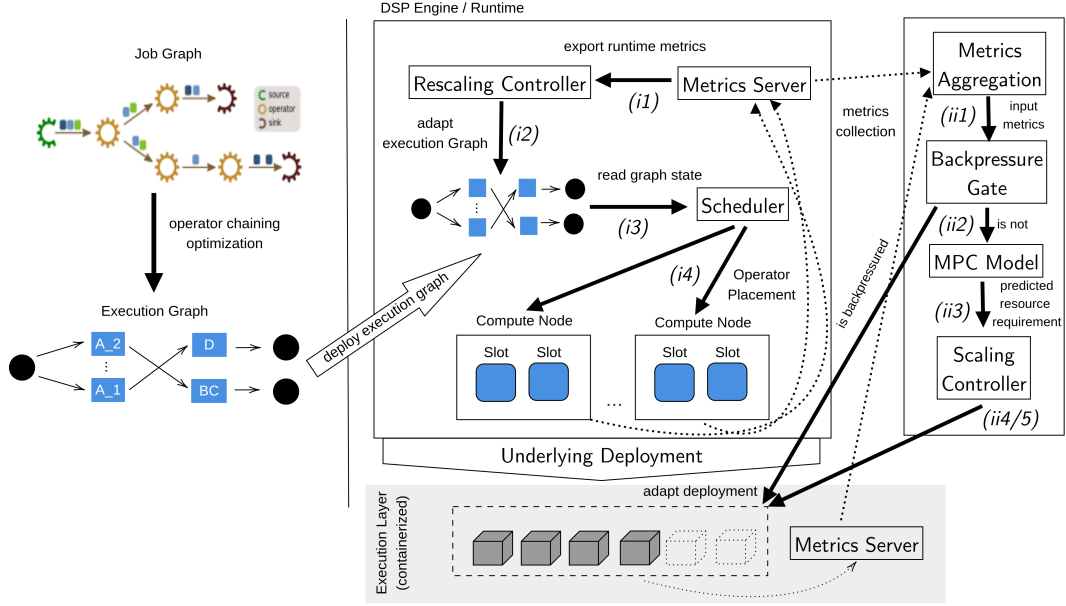


Figure 3.1: UpStream system architecture showing the layered scaling model and interaction between internal and external control loops.

3.3.1 Layered Scaling Model

The system is deployed on a containerized execution environment abstracted as a cluster of compute nodes, such as generic container workers or, in the case of Apache Flink, the task managers. On top of this environment runs a DSP engine, which executes the user-defined application graph. UpStream acts as an external controller that continuously monitors global application behavior and dynamically adjusts the number of available compute containers.

Each layer in the architecture plays a distinct role:

- **Execution Layer:** A pool of containerized workers that host task execution. Compute capacity is adjusted by scaling the number of containers.
- **DSP Engine Layer:** Executes the application's physical graph, manages task scheduling, and implements internal scaling by adapting job parallelism.
- **UpStream Controller:** Gathers metrics from the DSP engine, predicts future control actions using MPC, and orchestrates container scaling to

match resource demand.

3.3.2 (i) Internal Scaling Loop (DSP Engine)

Modern DSP engines are designed to be elastic. They include internal components such as a *scheduler* and a *rescaling controller* that continuously adapt the application's execution graph to changing resource availability and workload characteristics [14].

These components operate by (i1) monitoring internal runtime metrics and primarily resource availability. (i2) Adjusting operator parallelism in response to these metrics. This is achieved by increasing or decreasing the number of task instances per operator and redistributing state or partitioned streams accordingly. A key property of this internal controller of Apache Flink (reactive mode) is its **maximum utilization strategy**: it is assumed to consume *all available compute slots* to execute the application as efficiently as possible. Whenever new resources (e.g., containers) become available, the DSP engine's scheduler attempts to immediately scale out the operator graph to utilize those slots (i3 / i4). Conversely, if resources are reduced, a scale-down operation will be triggered to fit the new budget. This ensures that the DSP engine reacts promptly to any changes in compute capacity and maintains high resource utilization.

3.3.3 (ii) External Reactive / Predictive Scaling Loop (UpStream)

While the internal controller scales the application within the DSP engine, UpStream introduces a higher-level scaling loop that acts on the infrastructure level. It treats the DSP engine as a black box that always consumes all resources made available to it.

UpStream operates as follows:

- ii1 It continuously collects aggregated runtime metrics from the DSP engine (busy time, idle time and backpressure).
- ii2 Backpressure is used as a reactive gate before referring to the MPC model. If the system is backpressured, it reacts immediately and scales compute resources to alleviate the backpressured state.

- ii3* Busy and idle time are used as input to an MPC-based prediction model, which predicts future control decisions over a short time horizon.
- ii4* Based on this forecast, UpStream determines the optimal number of active task slots and compute nodes to meet performance goals while minimising resource waste.
- ii5* The controller then triggers a rescaling of the container pool, either by provisioning new nodes or releasing idle ones.

This loop complements the DSP engine’s internal elasticity: by adjusting the resource pool, UpStream aims to improve the performance of the DSP applications as well as resource utilization.

Together, the internal and external loops form a cohesive elasticity strategy. The DSP engine adapts operator parallelism to fully utilize available resources, and the UpStream controller adapts the amount of available resources based on its internal model.

This separation of concerns enables modular, platform-agnostic scaling. UpStream does not modify the execution graph directly. It focuses on scaling the infrastructure and delegates fine-grained task management to the DSP engine.

3.4 Control Loop at a Glance

Before the individual components are described in detail, this section summarizes how they fit together. At every synchronization period UpStream executes the loop in Algorithm 1. Each iteration has three stages:

1. Busy time μ_t , idle time ι_t and backpressure β_t are read from the DSP engine.
2. A scaling factor σ_t is determined. If backpressure exceeds the threshold β_{thr} , the reactive gate scales proportionally to β_t (Section 3.5). Otherwise, the MPC controller computes a smoothed scaling decision δ_t from busy and idle time (Algorithm 2). The factor is applied to the current allocation, which yields the target number of task slots S_{t+1} .
3. The greedy pod allocation (Algorithm 3) maps S_{t+1} onto nodes of different sizes, and the container pool is rescaled accordingly. The internal scaling

loop (labeled i in Figure 3.1) then adapts operator parallelism to the new slot count.

The number of task slots is the interface between the stages. The gate and the MPC only decide the number of slots needed and the allocation decides which compute nodes should be provisioned to fulfill that demand.

Algorithm 1 UpStream Control Loop (Overview)

Require: sync period Δ , threshold β_{thr} , node configurations, utilization factor u

```

1:  $S \leftarrow$  initial number of task slots
2: while true do ▷ once every  $\Delta$  seconds
3:    $(\mu_t, \iota_t, \beta_t) \leftarrow$  aggregated metrics of the DSP engine
4:   if  $\beta_t > \beta_{thr}$  then ▷ reactive gate (Sec. 3.5)
5:      $\sigma_t \leftarrow 1 + \beta_t$ 
6:   else ▷ predictive path (Sec. 3.6)
7:      $\delta_t \leftarrow$  first control input from Algorithm 2
8:      $\sigma_t \leftarrow 1 + \delta_t$ 
9:   end if
10:   $S' \leftarrow S \cdot \sigma_t$ 
11:  if  $S' \neq S$  then
12:     $nodes \leftarrow \text{PodAllocation}(S', \text{node configurations}, u)$  ▷ Algorithm 3
13:    rescale the container pool to  $nodes$ ;  $S \leftarrow$  provisioned slots
14:  end if
15:  wait until the next synchronization period
16: end while

```

3.5 Handling Backpressure

Backpressure is an indicator for underprovisioning. If we see backpressure in our pipeline, some operator in the DAG cannot keep up with its upstream input rate, causing buffers to fill and stalling events backwards through the pipeline. Unlike busy or idle time, backpressure is not well-suited as an MPC target metric because it is fundamentally asymmetric. An overprovisioned system will simply report zero backpressure regardless of how much spare capacity it has, so there is no meaningful "target value" to regulate towards. Backpressure can be reduced to zero in many different configurations, but it will never be negative to signal overprovisioning, the way busy and idle time can. Including it directly in the cost

function of the MPC model would therefore bias the controller without providing a useful gradient.

For this reason, UpStream treats backpressure as a *reactive gate* that is evaluated *before* the MPC component is invoked, rather than as one of its optimized metrics. At each control step, the observed backpressure β is compared against a fixed threshold. β_{thr} . Here, β is a fraction in $[0, 1]$, obtained by normalizing a **backpressured milliseconds per second** metric reported in the range $[0, 1000]$ by dividing by 1000 and β_{thr} is a target threshold, for example 0.01:

- If $\beta \leq \beta_{\text{thr}}$, backpressure is considered negligible, and control is delegated entirely to the MPC model described in Section 3.6, which balances busy and idle time over the prediction horizon.
- If $\beta > \beta_{\text{thr}}$, the MPC prediction step is skipped for that cycle, and the controller immediately scales the deployment in proportion to the severity of the backpressure.

This bypass trades away the smoothness and foresight of the MPC controller in exchange for responsiveness: backpressure spikes are typically caused by sudden bursts in input rate that the prediction model, operating on a short horizon of busy and idle time, can't anticipate.

3.5.1 Proportional Scaling Factor

The scaling factor σ_t applied at the control step t is computed as a linear function of the observed backpressure β_t :

$$\sigma_t = \begin{cases} 1 + \beta_t, & \text{if } \beta_t > \beta_{\text{thr}} \\ \sigma_t^{\text{MPC}}, & \text{otherwise} \end{cases} \quad (3.1)$$

where σ_t^{MPC} denotes the scaling factor produced by the MPC controller (Section 3.6) when backpressure is within acceptable bounds, and β_{thr} is an empirically chosen threshold (e.g. $\beta_{\text{thr}} = 0.01$) below which backpressure is treated as noise rather than a genuine bottleneck.

The resulting number of task slots S_{t+1} to provision for the next control interval is then obtained by scaling the current allocation S_t :

$$S_{t+1} = \lceil S_t \cdot \sigma_t \rceil, \quad S_{t+1} \geq 1 \quad (3.2)$$

The ceiling operator in Equation 3.2 ensures that any detected backpressure, however small, results in the provisioning of at least one additional task slot. It is preferable to briefly overprovision than to let backpressure persist. The lower bound $S_{t+1} \geq 1$ ensures that the deployment never shrinks below a single task slot.

Because σ_t grows linearly with β_t , larger backpressure values trigger proportionally larger scaling factors. For instance, a system reporting 50% backpressure ($\beta_t = 0.5$) will scale its task slot count by a factor of 1.5, while a system at the threshold of 10% scales only marginally, avoiding unnecessary noise close to the boundary between the reactive and predictive actions.

Note that the three quantities monitored by UpStream at each control step are not independent: busy time μ_t , idle time ι_t , and backpressure β_t jointly represent the total processing time of a task, such that:

$$\mu_t + \iota_t + \beta_t = 1. \quad (3.3)$$

This relationship motivates the layered design of the controller: the MPC component described is only ever invoked when $\beta_t \leq \beta_{thr}$, i.e. precisely when, backpressure is negligible. In this regime, Equation 3.3 reduces to $\mu_t + \iota_t \approx 1$, which allows modelling the system state using only the two dominant components, busy time and idle time, in the prediction model used by the MPC controller.

3.6 Model Predictive Control for Resource Scaling

A main contribution of UpStream lies in its use of a Model Predictive Control (MPC) strategy to achieve proactive and workload-aware scaling. While traditional DSP resource managers react to observed conditions, MPC enables anticipation of future resource demands by explicitly modelling system dynamics and optimizing control actions over a defined prediction horizon.

MPC operates by first observing the current system state captured through a set of internal metrics. Using a model of the system, it then forecasts how these metrics will evolve over a finite future horizon, given possible control inputs (in

this case, scaling decisions). Based on these predictions, the controller formulates and solves an optimization problem to determine the best course of action that minimizes a defined cost function.

The general structure of the cost function used in UpStream is quadratic and multi-objective. It is defined as:

$$\begin{aligned}
 J = & \sum_{t=0}^{T-1} \sum_{m=1}^M w_m \cdot \left(\hat{M}_{m,t} - M_m^{\text{target}} \right)^2 \\
 & + \sum_{t=0}^{T-1} \left[\lambda_1 \left(\hat{\delta}_t - \hat{\delta}_{t-1} \right)^2 + \lambda_2 \hat{\delta}_t^2 \right] \\
 & + \sum_{m=1}^M w_m \cdot \left(\hat{M}_{m,T} - M_m^{\text{target}} \right)^2
 \end{aligned} \tag{3.4}$$

The last term of this sum is pulled out of the first sum to be consistent with the expression in the `do-mpc`¹ library used in the implementation of UpStream. Here, $\hat{M}_{m,t}$ denotes the predicted value of the performance metric m at the future time step t , while M_m^{target} represents its desired value. Each metric is weighted by a factor w_m reflecting its relative importance. The second term penalizes abrupt changes in the control input δ_t , with the regularization factor λ_1 promoting smoother adjustments to cluster size and λ_2 penalizing high values of δ_t .

Although the optimization yields a complete sequence of future control actions, only the first value is enacted in practice. This receding horizon principle ensures that the controller continuously reacts to updated observations and predictions. In UpStream, the control input δ_t corresponds to a scaling decision for the number of compute containers to be deployed in the cluster. For implementation purposes, this control input is often represented in normalized form as a deviation from the current resource allocation:

$$p_t = p_{\text{current}} \cdot (1 + \delta_t)$$

The value δ_t is the effective output of the controller and encodes the recommended scaling adjustment, derived from minimising the cost function J . It can be interpreted as a deviation term relative to the current capacity, guiding whether to increase or decrease resource allocations in the next scheduling cycle. By con-

¹<https://www.do-mpc.com/en/latest/index.html>

tinuously updating predictions and control inputs, the MPC controller ensures responsiveness to changes in workload intensity while still smoothing the increase and decrease of resources to reduce the impact of the scaling decision.

3.6.1 System Dynamics Model

To predict how the monitored metrics evolve under a given control action, Upstream uses a discrete-time linear state-space model. The system state at control step t is given by the vector:

$$x_t = \begin{bmatrix} \mu_t \\ \iota_t \end{bmatrix}, \quad (3.5)$$

containing only busy time μ_t and idle time ι_t . Limiting the state to these two components is justified by the fact that busy time and idle time dominate the execution time of a task and can be assumed to be complementary once backpressure has been filtered out by the reactive gate of equation 3.1.

The control input δ_t (section 3.6) enters the model through the following linear approximation of the state transition:

$$\mu_{t+1} = \mu_t - \kappa \cdot \delta_t, \quad (3.6)$$

$$\iota_{t+1} = \iota_t + \kappa \cdot \delta_t, \quad (3.7)$$

where κ is a fixed control-authority coefficient that determines the sensitivity of the predicted state transition to a given scaling decision. Intuitively, a positive deviation δ_t (scaling out, i.e. provisioning additional task slots) spreads the same workload across more compute capacity, resulting in busy time decreasing and idle time increasing by a corresponding amount, and vice versa for a negative deviation. Note that the two equations are coupled by construction, so that $\mu_{t+1} + \iota_{t+1} = \mu_t + \iota_t$ for any δ_t : the model redistributes time between the two states without changing their sum, consistent with the near-complementarity assumption above.

In contrast to a conventional state-space model, Equations 3.6 and 3.7 contain no autonomous term. The predicted state remains constant rather than evolving on its own. The model therefore does not attempt to capture the intrinsic dynamics of the workload (e.g. organic load growth or decay between control steps). It only expresses how the controller expects the state to move in response to its own ac-

tions. Workload-driven changes to μ_t and ι_t are instead captured indirectly, since the state is reset to the freshly measured values μ_t, ι_t at the start of every control step before a new trajectory is optimized (Algorithm 2). This re-initialization at every step allows the controller to remain responsive to external load changes despite the purely control-driven prediction model.

3.6.2 Implemented Cost Function

To apply the MPC control strategy, UpStream optimizes over the two domain-specific state metrics introduced in Section 3.6.1:

- ι_t : predicted `idle_time` at time t
- μ_t : predicted `busy_time` at time t

The control objective is to drive both metrics toward their respective target values, ι^* and μ^* , while limiting control effort and abrupt changes in the control input. The resulting cost function over a prediction horizon T is formulated as:

$$\begin{aligned}
 J = & \sum_{t=0}^{T-1} \left[\alpha_1 (\iota_t - \iota^*)^2 + \alpha_2 (\mu_t - \mu^*)^2 \right] \\
 & + \sum_{t=0}^{T-1} \left[\lambda_1 (\hat{\delta}_t - \hat{\delta}_{t-1})^2 + \lambda_2 \hat{\delta}_t^2 \right] \\
 & + \alpha_1 (\iota_T - \iota^*)^2 + \alpha_2 (\mu_T - \mu^*)^2
 \end{aligned} \tag{3.8}$$

Here, α_1 and α_2 are weighting coefficients controlling the influence of the idle-time and busy-time tracking errors, respectively, and are both chosen to be 1 since these two metrics directly correlate: reducing the deviation of one tends to reduce the deviation of the other. The term weighted by λ_2 penalizes control effort directly. In the current configuration $\lambda_2 = 0$, effectively disabling this term so that control effort is constrained only through the smoothness penalty. λ_1 penalizes large fluctuations in the control input between consecutive steps and $\hat{\delta}_t$ is the predicted scaling deviation applied at time t .

This cost function ensures that the system converges toward its target busy and idle time fractions, μ^* and ι^* . The smoothness term encourages gradual scaling actions, avoiding instability due to aggressive changes in cluster size.

In addition to the running cost accumulated over the horizon, the cost function includes a terminal cost, evaluated once on the state predicted at the end of the horizon ($t = T$), separately from the per-step tracking cost. This term plays a distinct role from the stage cost: because the horizon is finite, the optimizer only "sees" the stage cost up to $t = T - 1$, which can leave it free to neglect the state it predicts at the very edge of the horizon, since no stage cost is ever charged there. The terminal cost closes this gap by explicitly penalizing deviation from ι^* and μ^* at $t = T$ as well, biasing the optimizer toward trajectories that not only track the targets throughout the horizon but specifically end close to them.

In the present implementation, the terminal cost reuses the identical tracking-error expression used in the stage cost, i.e. the same $\alpha_1(\iota_t - \iota^*)^2 + \alpha_2(\mu_t - \mu^*)^2$ form is evaluated both at every intermediate step and, once more, at $t = T$. The two are not functionally distinct in this configuration. The terminal cost simply adds extra weight to hitting the targets by the end of the prediction window, rather than expressing a separate terminal condition.

Minimizing J at each control interval yields the optimal resource allocation decision. The controller applies only the first action of the optimized sequence (receding horizon strategy), repeating the process at the next timestep with fresh observations.

Algorithm 2 MPC-Based Scaling Controller

- 1: **Input:** Current metrics ι_t, μ_t , target values ι^*, μ^* , prediction horizon T , weights $\alpha_1, \alpha_2, \lambda_1, \lambda_2$, control-authority coefficient κ , current control input δ_{t-1}
- 2: **Output:** Scaling decision δ_t
- 3: **for** each control step t **do**
- 4: Predict metric trajectories over horizon T :

$$\boldsymbol{\iota}_{t\dots t+T-1}, \boldsymbol{\mu}_{t\dots t+T-1}$$

- 5: Evaluate cost function J ▷ Equation 3.8
- 6: Solve: $\min_{\delta_{t\dots t+T-1}} J$
- 7: Apply first control input:

$$\delta_t \leftarrow \delta_t^{\text{opt}} \in \delta_{t\dots t+T-1}$$

- 8: **end for**
-

3.7 Compute Node Allocation

Given the output of steps (ii2 or ii3) of section 3.3.3, the next step is to decide how the task slots should be allocated to containers of different sizes to reduce the overhead of purely horizontal scaling.

This approach employs a greedy algorithm to allocate the required task slots to the different compute node configurations in the container deployment of the DSP engine. The algorithm prioritizes larger nodes with more task slots if the required number of slots is large enough to necessitate the use of a pod of that particular size. This helps to reduce overhead by favoring fewer, larger nodes rather than scaling with many small ones, which can introduce unnecessary complexity and resource waste and add network latency due to inter-node communication requiring the whole networking stack.

Algorithm 3 Pod Allocation Algorithm

```

1: Input: number_of_slots  $\geq 1$ , nodes (list of node configurations), utilization_factor
2: Output: allocated_nodes (list of nodes with assigned replica counts)
3: Sort the nodes based on task_slot_capacity in descending order.
4: Initialize allocated_nodes as an empty list.
5: for each node in sorted_nodes do
6:   node.replica_count  $\leftarrow 0$ 
7:   while number_of_slots  $\geq \text{round}(\text{node.task\_slot\_capacity} * \text{utilization\_factor})$  do
8:     number_of_slots -= node.task_slot_capacity
9:     node.replica_count += 1
10:  end while
11:  Append node to allocated_nodes
12: end for
13: if number_of_slots  $> 0$  then
14:   Increment replica_count of the last node in allocated_nodes
15: end if
16: Return allocated_nodes

```

One implementation detail of this algorithm is the *utilization_factor*. It plays a critical role in the node allocation process. It determines the degree to which the task slots of a node are utilized before a new replica is instantiated. The *utilization_factor* helps in avoiding underutilization of resources, which can lead to the creation of unnecessary pods. By adjusting this parameter, we can balance the need for efficient resource allocation with the requirement to maintain elasticity. For example:

- A *utilization_factor* of 1.0 means that a node will be fully utilized to its maximum capacity before a new node is instantiated. In this case, the system aims to fill each node as much as possible before scaling horizontally.
- A *utilization_factor* less than 1.0 represents a utilization below 100%. A utilization factor of 0.8 indicates that each node will be filled only up to 80% of its capacity before scaling occurs, thereby leading to more nodes being allocated but with lower resource utilization per node.

It should be noted that the implementation of Algorithm 3 used throughout this thesis contains an inconsistency between its loop condition and its update step: the **while do** condition on line 7 compares *number_of_slots* against the utilization-factor-scaled capacity `round(node.task_slot_capacity * utilization_factor)`, but line 8 subtracts the *unscaled* `node.task_slot_capacity` from *number_of_slots*. For *utilization_factor* = 1.0 these two quantities coincide and the algorithm behaves as intended. For *utilization_factor* < 1.0, however, this decouples the allocation decision from the actual demand being served, so the algorithm no longer guarantees that each node is loaded only up to the configured fraction of its capacity. Since all results reported in this thesis were produced with *utilization_factor* = 0.8 (Section 6.4), the reported node allocations reflect this implementation artifact. Section 7.3 gives the corrected version, in which line 7 and line 8 both use $\text{cap}_{\text{eff}} = \text{round}(\text{capacity} \cdot u)$. Note that with $u < 1$ the allocated slots may exceed the demand by design.

4 Stream Processing Applications

To evaluate the performance of the UpStream autoscaler under realistic conditions, this thesis utilizes a suite of applications sourced from the RIoT Bench IoT benchmark proposed by Shukla et al. [34].

The original RIoT Bench applications were implemented for the Apache Spark¹ data processing engine. In this work, they were adapted and implemented for the Apache Flink² [5] stream processing engine using the Apache Beam³ framework. Apache Beam is an open-source, unified programming model that allows both batch and streaming data pipelines to be defined using a single API, independent of the underlying execution engine. Pipelines defined with Beam’s model are portable across multiple execution engines including Apache Flink, Apache Spark, and Google Cloud Dataflow. The applications ETL and STATS were implemented by the author, while PRED and TRAIN were implemented by another Bachelor student, Jona Müller. The source code for all applications is available on GitHub⁴.

4.1 Application Architecture

The benchmarking application are implemented using Apache Beam in Java and designed as a distributed stream processing pipeline. The pipelines interact with Kafka topics for data ingestion and inter-pipeline communication. A MongoDB instance provides persistent storage of processed results. While the experimental deployment utilizes containerized components orchestrated in a Kubernetes environment, the overall architecture remains platform-agnostic and can be executed

¹<https://spark.apache.org/>

²<https://flink.apache.org/>

³<https://beam.apache.org/>

⁴<https://github.com/DPS-Stream-Processing-Applications/beam-applications-java/tree/riotbenchsinglejob>

in any compatible distributed environment.

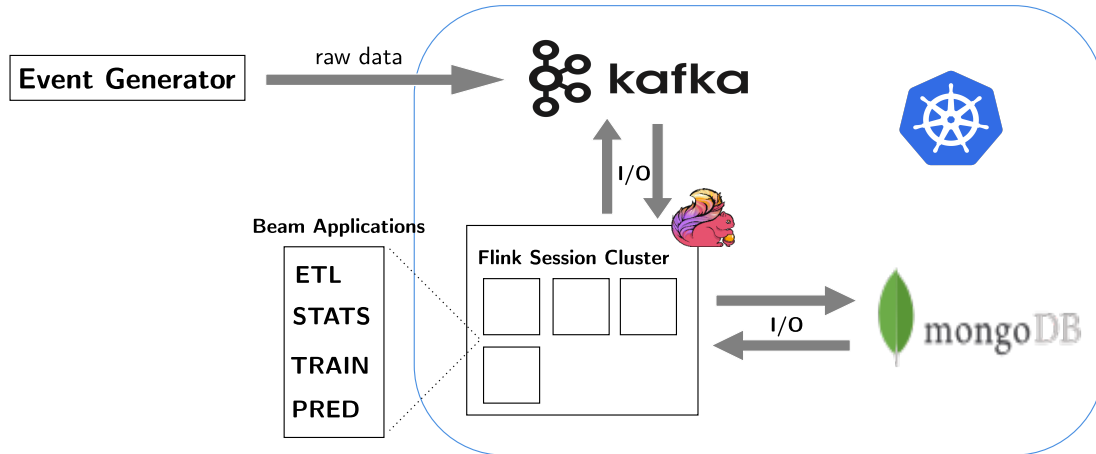


Figure 4.1: RIoT application architecture

4.2 Pipelines

RIoTBench implements four individual data pipelines that ideally depend on each other but can also be deployed standalone. Each of these pipelines is comprised of multiple processing tasks. Each operator and their applied transformations are collected in table 4.1.

4.2.1 ETL

This Extract-Transform-Load job performs data cleaning and interpolation to preprocess the raw data for later use by other stream processing jobs. This pipeline does not contain any conditional branching, meaning every event follows through the same sequence of transformations. This uniform cost should allow the pipeline's throughput to scale near-linearly with allocated parallelism, without the risk of branch-dependent processing imbalances that can limit scalability in more complex pipelines.

4.2.2 STATS

This job performs data aggregation through the Average and Interpolate transforms as well as some noise reduction through the use of a Kalman Filter and

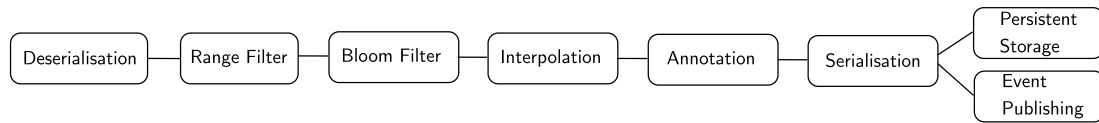


Figure 4.2: Transformations of the ETL pipeline in order

visualizes all output streams as plots to be stored for later review.

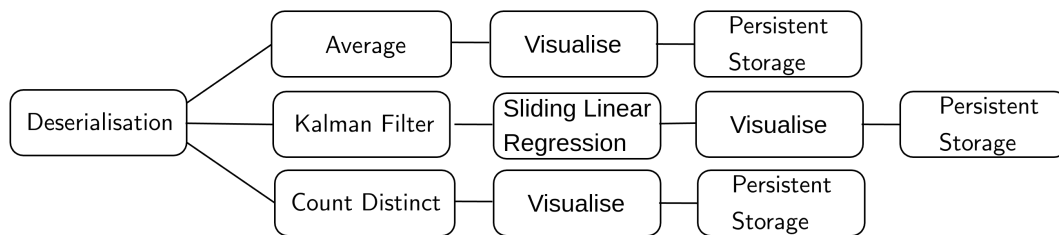


Figure 4.3: Transformations of the STATS pipeline in order

4.2.3 TRAIN

This job trains two machine learning models: the linear regression model and the decision tree model. Training data is sourced from MongoDB, and the trained models are also stored on MongoDB. At the final stage of the pipeline, a message is published to the `train-publish` Kafka topic, signalling that new models are available.

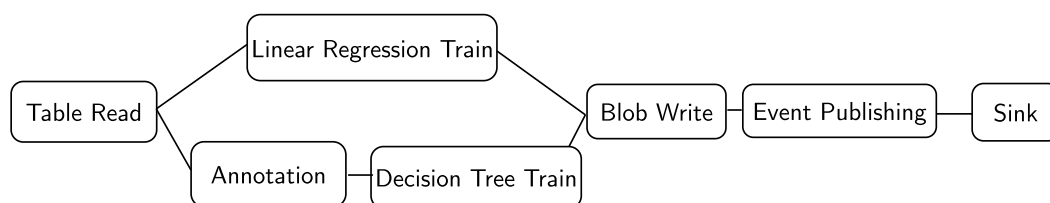


Figure 4.4: Training process of TRAIN pipeline

4.2.4 PRED

This job performs predictions on input records using the models produced by the TRAIN job. It calculates the average and provides a residual error estimation for

each prediction. As the final stage of the pipeline, it publishes the predictions to the `pred-publish` Kafka topic.

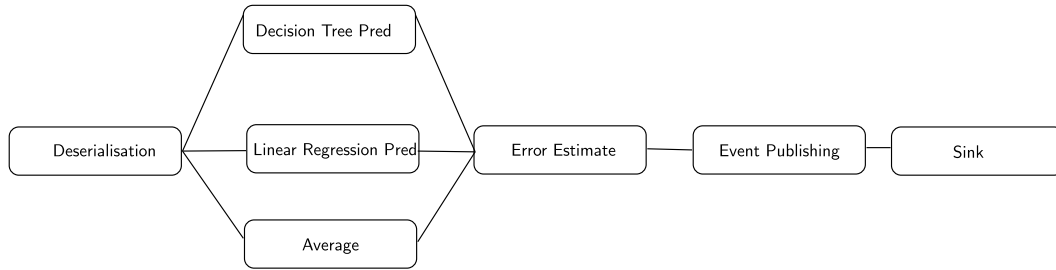


Figure 4.5: Predictive analysis

4.3 Summary of the Application Suite

The suite of pipelines collectively forms a robust stream processing application designed for real-time data workflows. Starting with raw data ingestion, the ETL pipeline ensures data integrity through cleaning and interpolation. The processed data is then analysed by the STATS pipeline, which aggregates statistics and visualises trends. The TRAIN pipeline develops machine learning models on a scheduled basis, leveraging historical data from MongoDB, and the PRED pipeline applies these models to perform real-time forecasting and error estimation.

The pipeline interaction, depicted in Figure 4.6, demonstrates a complete data lifecycle. This design transforms unstructured raw input into actionable insights, supporting various analytical and predictive use cases.

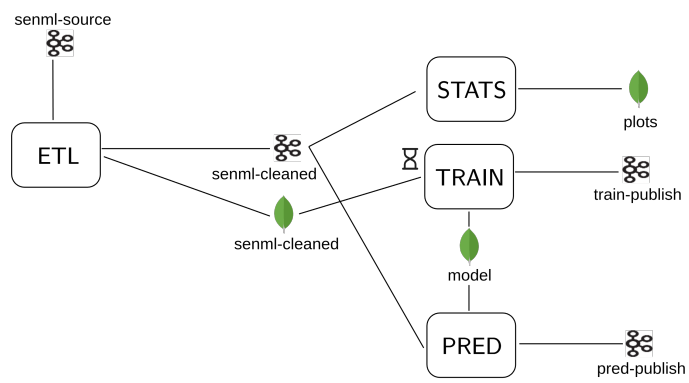


Figure 4.6: Interaction between RIoT Bench pipelines

Transform	Description
Deserialisation	Parses the customized SenML format into a Java object. All following operations are performed on this internal data representation.
Range Filter	Sets data points to <code>null</code> based on fixed boundaries for each dataset. These nullified data points are interpolated downstream.
Bloom Filter	A Bloom Filter [4] validates that all data points originate from expected sources. The ID of the event source determines whether the data point should be dropped.
Interpolation	Interpolates missing data points within a fixed window by computing their mean value.
Annotation	Adds additional derived data that was not present in the original event.
Table Read	Accesses reference or historical data from MongoDB for model training or validation.
Linear Regression Train/Pred	Trains or applies a linear regression model to predict values based on historical patterns in the input stream.
Decision Tree Train/Pred	Trains or applies a decision tree model to perform classification or regression on the input data.
Error Estimate	Estimates the residual error between observed and predicted values to assess model accuracy.
Serialisation	Serialises the internal data representation into SenML format for ingestion by subsequent applications.
Average	Computes the average value of a field over a defined time window.
Count Distinct	Calculates the number of distinct events (based on ID) within a fixed attention window.
Kalman Filter	Applies a Kalman Filter [23] to smooth the input stream and reduce statistical noise using state prediction.
Sliding Linear Regression	Performs linear regression on the input stream within a fixed attention window.
Visualise	Generates byte arrays of plots for the input stream using custom plotting functions for each data type.
Persistent Storage	Persists events and processed data in MongoDB for access by non-realtime applications.
Event Publishing	Publishes processed events to a new Kafka topic for further downstream processing.

Table 4.1: Transformations of the different jobs

5 Datasets and Workloads

All of the applications of the RIoT benchmark ingest events adhering to the Sensor Measurement List (SenML)¹ data format. SenML is standardized in RFC 8428 [18], and aims to be a lightweight data format for representing sensor readings and device parameters. A SenML **pack** consists of an ordered list of **records**, each describing a single measurement through a small set of fields:

- A name identifying the sensor or data stream (**n**);
- An optional unit (**u**);
- A value, given as **v**, **vs**, **vb**, or **vd** depending on whether it is numeric, string, boolean, or binary data;
- A timestamp (**t**).

To reduce redundancy, records can share common fields such as a base name, base unit, or base time, which are specified once at the start of the pack and applied to all subsequent records unless overridden. Although SenML is most commonly serialized as JSON, RFC 8428 also defines equivalent CBOR, XML, and EXI representations that share the same underlying data model. The IANA registry of standardized field names and units is maintained at <https://www.iana.org/assignments/senml> [15].

SenMLConverter Useful datasets such as the ones described in section 5.1 are not natively available in the SenML format. For this purpose, the raw data must first be converted and then rescaled to different load profiles (section 5.2) for our evaluation. A small Python script was implemented that ingests the raw datasets and generates a SenML-formatted CSV file for further processing.

¹<https://www.iana.org/assignments/senml>

5.1 Datasets

The RIoT pipelines were implemented to ingest three different datasets. All of them get the same kind of computations applied to their data once converted to SenML format.

5.1.1 TAXI

TAXI is a dataset tracking taxi routes and purchases throughout multiple weeks. It was chosen due to its seasonality in the number of purchases. For the evaluation, data from the University of Illinois ² were used as the source [9].

5.1.2 FIT

The FIT dataset is derived from the MHEALTH dataset [3], which contains body motion and vital sign recordings of volunteers performing physical activities. Sensors attached to the subject's chest, right wrist, and left ankle capture the motion of different body parts through acceleration, rate of turn, and magnetic field orientation. Additionally, a chest-mounted sensor records a 2-lead ECG signal. This dataset can be seen as an example of using real-time stream processing to track metrics of athletes during a competition.

5.1.3 GRID

GRID is a confidential CER electricity dataset containing energy usage data from multiple households over an extended time period. It is not publicly available.

5.2 Load Profiles

To evaluate the scaler, three load profiles are used across the experiments, driven by a custom event generator. The generator produces a configurable stream of SenML-formatted events to the Kafka source topic, with the event rate shaped according to the profile.

²<https://doi.org/10.13012/J8PN93H8>

5.2.1 Load Profile Generation

The load profiles used for the evaluation are configured as follows: Every profile takes a total of 15 minutes, during which data is actively pushed to the target Kafka topic. This is equivalent to 900,000 milliseconds. To convert our raw SenML CSV files from section 5 into our desired load profiles, two scripts were used. The `SenMLScenarioBuilder` generates the synthetic load profiles Constant Load and Exponential Ramp. It works by defining a probability density function of our desired form, sampling a certain number of data points from it between zero and 900,000 and matching the given millisecond sample with an event from the raw SenML data of section 5. The number of samples is calculated by the total time the given scenario is running (15 minutes) as well as a target events-per-second rate. The specific load profiles used for the evaluation are described in the following sections.

5.2.2 Constant Load

A constant stream of 10,000 events per second. Due to the random sampling of the dataset from a PDF, we see a small noise spike of 11 events in one millisecond bucket.

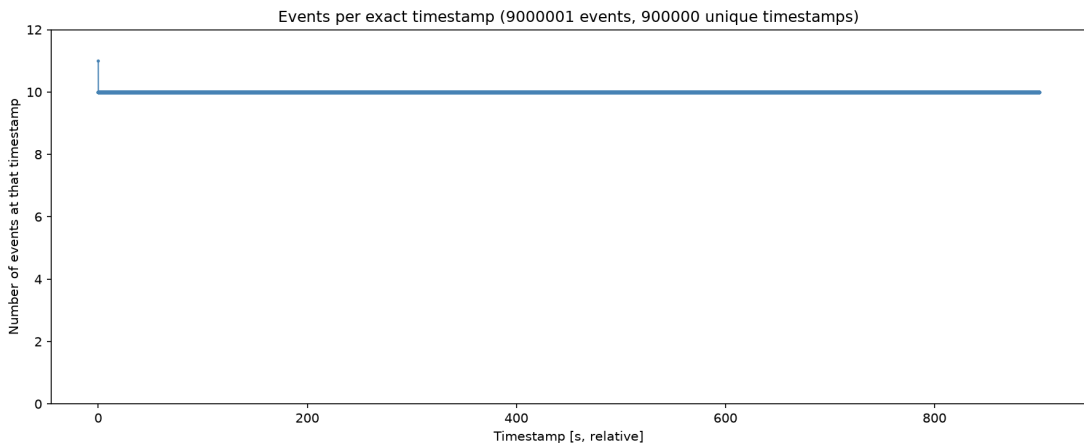


Figure 5.1: Constant rate scenario with ten events per millisecond, meaning 10,000 events per second

5.2.3 Exponential Ramp

The event timestamps were generated from a simple two-part shape over the 15-minute scenario duration T : the arrival rate grows exponentially for the first $t_1 = 10$ minutes, then stays flat at its peak value for the remaining 5 minutes. This shape is described by the unnormalized function

$$f(t) = \begin{cases} e^{t/\tau} - 1, & 0 \leq t \leq t_1, \\ e^{t_1/\tau} - 1, & t_1 < t \leq T, \end{cases} \quad (5.1)$$

where $\tau = t_1/3$ controls how quickly the rise happens, and the plateau simply continues at the value reached at the end of the rise. To make the rate function independent of the maximum number of events, we normalize, resulting in a probability density function (PDF) of the form:

$$p(t) = \frac{f(t)}{\int_0^T f(s) ds}. \quad (5.2)$$

This has the advantage that the corresponding cumulative distribution function (CDF),

$$F(t) = \int_0^t p(s) ds, \quad (5.3)$$

allows us to sample independently of the unit of time by sampling timestamps from its inverse F^{-1} .

This makes the output reproducible independent of the chosen number of samples or unit of time across runs while still following the intended load shape. This profile is specifically designed to stress reactive baselines: Because the rate of change is itself increasing, a scaler that responds only to current utilization will consistently lag behind the load curve.

5.2.4 Realistic TAXI Profile

The raw TAXI dataset 5.1.1 provides a realistic, irregular load distribution with natural temporal variation, including intra-day peaks and weekend patterns. This is also called seasonality. Unlike the synthetic profiles, the irregularity is precisely what makes it valuable for testing.

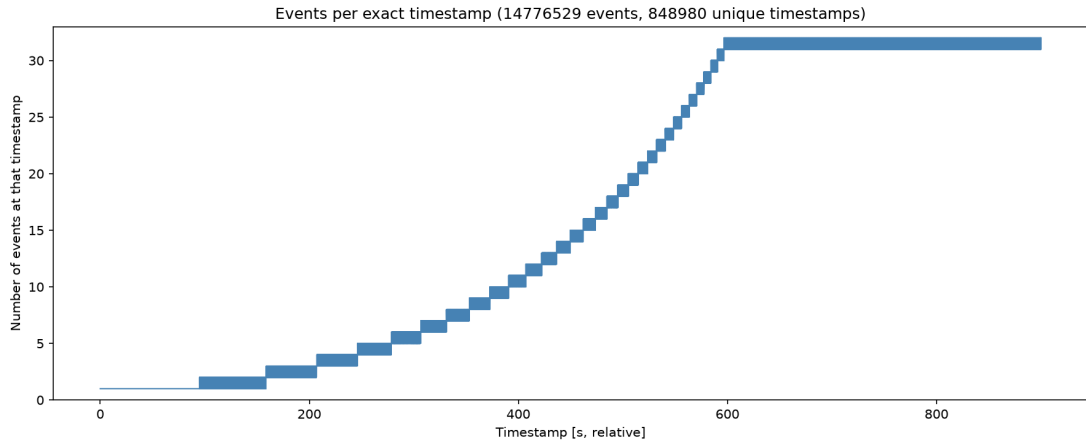


Figure 5.2: Exponential ramp scenario with a maximum of thirty events per millisecond, meaning 30,000 events per second

The events used for this profile were handpicked from a daily distribution. This time window was then scaled down to fit the 15-minute window used for the profiles, and the number of events per second was scaled to have a max of 10,000 events using a small Python script.

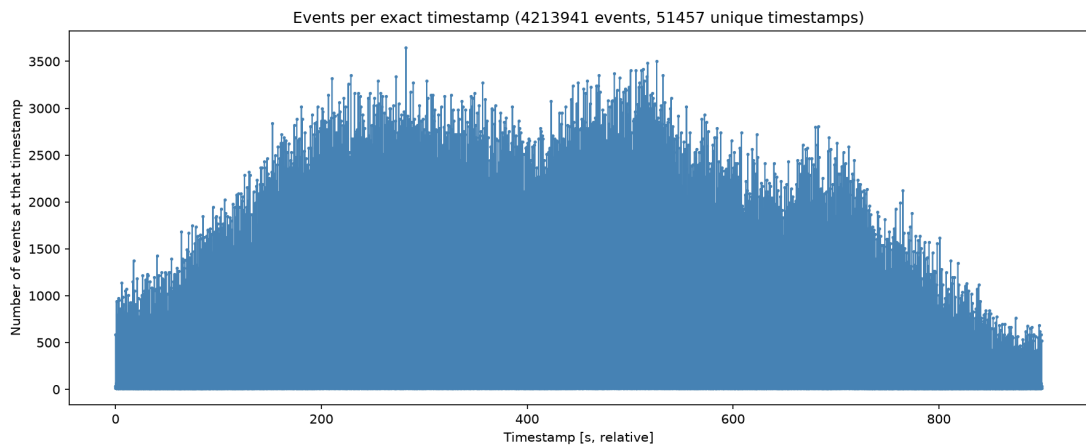


Figure 5.3: TAXI data sample of a single day, scaled to have a maximum of 10,000 events within a window of a second and all other buckets scaled accordingly.

5.3 Event Generator

To evaluate the scaler, we need reproducible load profiles emitted to the source topics of our applications. This section describes the generator application that was implemented to produce said events by ingesting the load profiles from section 5.2.

The event generator is a multi-threaded, timing-based Kafka producer that replays a recorded event log from a CSV file. On startup, the `Main` class spawns a configurable number of worker threads, each running an event producer loop.

Rather than reading the file row by row and passing it as tasks to a thread-pool, the workload is striped row-by-row across threads: thread i begins at row i and, after each read, skips forward by `STRIDE` rows (equal to the thread count), so thread 0 consumes rows $0, N, 2N, \dots$, thread 1 consumes rows $1, N+1, 2N+1, \dots$, and so on. This partitioning lets the threads operate independently and in parallel without contention over shared file state or a task queue. Each CSV row encodes an elapsed time (in milliseconds, relative to a shared start timestamp) and an event. The owning thread busy-waits until this target time has elapsed before publishing the payload to the configured Kafka topic. The result is a faithful, time-accurate replay of the original event trace, produced concurrently by multiple threads.

This approach is limited by the number of threads running concurrently. If more than N events must be emitted at the same timestep, only N of them can be published in parallel. Any additional events sharing that timestep are queued behind a thread's own row sequence and are only published once that thread finishes its current row and reads the next one. This introduces a small delay relative to the original timestamps. In practice, this overhead is minimal, since each thread progresses independently and is slowed only by the I/O cost of publishing the current event and reading the next row from the file.

6 Evaluation and Results

6.1 Infrastructure Setup

The bare-metal compute infrastructure used in this thesis was provisioned using the Chameleon Cloud [24] research testbed. All resources were provisioned in the CHI@TACC region.

The experimental setup comprises four dedicated physical servers: one `compute_skylake` node functioning as an SSH bastion host and deployment target for the multi-threaded Event Generator (Section 5.3), and three `compute_cascadelake` nodes hosting the containerized execution environment.

All compute nodes are interconnected via standard high-speed cluster networking (`sharednet1`) to minimise latency during event replay and stream transmission. Table 6.1 details the specific hardware configurations. It is important to note that `sharednet1` is a shared network interconnect, meaning nodes are exposed to unrelated network traffic and noisy neighbour effects. This is intended since realistic cloud deployments will usually also experience some level of network noise.

Attribute	Cascade Lake Node	Skylake Node
Node type	<code>compute_cascadelake</code>	<code>compute_skylake</code>
Site	TACC	TACC
Architecture	x86_64	x86_64
CPU	Intel Xeon Gold 6242 @ 2.80 GHz	Intel Xeon Gold 6126 @ 2.60 GHz
Threads	96	48
RAM	192 GiB	192 GiB
Total storage	200 GB	200 GB
Network	<code>sharednet1</code>	<code>sharednet1</code>
OS	Ubuntu LTS 26.04	Ubuntu LTS 26.04

Table 6.1: Hardware configuration of the Chameleon Cloud compute nodes used.

The three `compute_cascadelake` nodes host a Kubernetes cluster. To set this up,

the lightweight Kubernetes distribution K3S¹ version 1.30.2 is deployed via the officially maintained Ansible playbook². One node hosts the k3s-server component while the other two nodes are configured as worker nodes (k3s-agent) connected to the server node. Pod networking and inter-container communication were managed via the default Flannel CNI over containerd provided by K3S. Unlike some setups that isolate the control plane on untainted control-plane nodes, no node taints were applied to the server node in this environment. Consequently, all three bare-metal nodes actively host deployed application pods, allowing for the maximization of compute utilization across the available hardware.

It is important to distinguish between infrastructure node elasticity and application container elasticity: the underlying physical node count remained static for the entire deployment lifecycle. Autoscaling mechanisms evaluated in this work operate strictly at the container orchestration layer by dynamically scaling pod replicas across the provisioned cluster nodes.

6.2 Application Deployment

The RIoT Bench applications have a number of dependencies, as already mentioned in section 4.1. All components to be deployed are packaged with the Helm³ package manager. In total, there are four charts:

- **mongodb**: MongoDB is the document store database used by the applications. The Helm chart used installs the official Docker image with MongoDB version 6.0.
- **kafka**: Kafka is the message queue used to deliver the events processed by the pipelines. The Helm chart used uses the official Strimzi community-maintained chart and configures Kafka to be deployed with three brokers that all use local ephemeral storage on the nodes they are running on. Due to the short-lived experimental scenarios, we do not require long-term persistent storage.

¹<https://k3s.io/>

²<https://github.com/k3s-io/k3s-ansible>

³<https://helm.sh/>

- **monitoring:** For tracking all the performance metrics as well as the interface for our UpStream scaler, we use Prometheus deployed via the kube-prometheus-stack Helm chart that deploys the Prometheus metric collector as well as Grafana for easy visualization of our target metrics. For this purpose, a custom dashboard was created and exported as JSON to be imported into Grafana manually after deployment.
- **flink:** The core DSP engine used for our evaluation is Apache Flink version 1.20. The custom Apache Flink Helm chart, which depends on the official one provided by the Apache project, provides two value files to deploy the baseline scaler as well as the configuration needed for the UpStream implementation. The main feature that both approaches rely on is the Apache Flink Reactive Mode. It automatically scales the application DAG after a change in the underlying number of task managers occurs. The example deployment of Apache Flink using Reactive Mode requires checkpointing to be enabled. During initial testing, we ran into significant issues with Kafka source tasks not scaling appropriately with Reactive Mode and checkpointing enabled. After lengthy trial and error, the only feasible fix was to remove checkpointing from the deployment and have Reactive Mode enabled standalone. This means that on a scaling of the DAG, any state that was stored within Tasks is lost, including events currently in the pipeline. This is far from ideal, but it applies to both the baseline and UpStream deployments, meaning comparability of the two is still given.

Apache Flink is deployed as an Application Cluster, meaning the Apache Flink deployment also packages the Job JAR file of the pipelines described in section 4.2 in its deployment. The JAR artifacts were uploaded to the Kubernetes nodes manually beforehand to make them available through a file mount within the deployed containers.

To evaluate the performance and efficiency of UpStream, we conduct a comparative empirical evaluation against the standard Kubernetes Horizontal Pod Autoscaler (HPA). This chapter details the autoscaler configurations, the experimental matrix, the evaluated metrics, and the execution protocol.

6.3 Scaler Configuration

We evaluate two distinct auto-scaling approaches:

- **Kubernetes HPA (Baseline):** A standard reactive autoscaler configured to monitor aggregate CPU and memory utilization. Scaling decisions are triggered when average utilization exceeds 80%. Under HPA, the cluster scales horizontally using a single Task Manager pod configuration (Small: 2 task slots per Task Manager).
- **UpStream (MPC + Node Allocation):** Our proposed framework utilizes Model Predictive Control over domain-specific metrics (busy time and idle time) combined with a reactive backpressure gate. Unlike HPA, UpStream employs the greedy node allocation algorithm (Section 3.6) to select dynamically across three task manager sizes: Small (2 slots), Medium (8 slots), and Large (16 slots). This allows UpStream to optimize resource packing and reduce inter-pod networking overhead as the cluster scales out.

Size	Task Slots	CPU Request/Limit	Memory
Small (Default)	2	1 Core	2 GiB
Medium	8	8 Cores	16 GiB
Large	16	16 Cores	32 GiB

Table 6.2: Task Manager Pod Configuration Sizes

6.4 Hyperparameters

The following configuration was used for HPA: The HPA targets the aggregate CPU and memory utilization of the default TaskManager deployment across a scaling range bounded by a minimum of one and a maximum of 64 replicas. 64 replicas have been shown to be the maximum number of replicas achievable on the three-node testbed during initial testing. With a replica count higher than 64, container crashes and error states were observed due to resource bottlenecks. Both target CPU utilization and target memory utilization are set to 80%, with

the HPA continuously computing target replica counts independently for each resource type and enacting whichever recommendation yields the larger cluster size. To adapt standard Kubernetes HPA behavior to real-time data stream processing demands, the default scaling dynamics were tuned for high responsiveness. Both scale-up and scale-down stabilization windows were reduced to 15 seconds, and the scaling rate policies were configured to permit a maximum change of up to 100% of current replicas every 15 seconds, allowing rapid resource expansion during sudden load spikes as well as immediate resource reclamation post-burst.

Although the native Kubernetes HPA was configured with a 15-second stabilization window to maximize reactivity, empirical observations revealed that the effective synchronization interval for both scalers is bound by Apache Flink’s internal rescaling overhead. Upon any modification to TaskManager pod replicas, Apache Flink’s Reactive Mode requires approximately 120 seconds tearing down the physical execution graph, re-registering compute slots, and resuming processing. To ensure stability and a fair comparison between mechanisms, the synchronization loop interval (Sync period) for UpStream was set to 120 seconds, matching the physical stabilization time observed with HPA.

The `do-mpc`⁴ [10] Python library is used to build and solve the MPC optimization problem. Beyond standard setpoint targets, `do-mpc` requires several framework-specific hyperparameters to manage optimization dynamics and solver stability:

- **Control authority (κ):** Scales the impact of control deviation inputs on discrete state transitions.
- **Control regularization (`r_term_control`, `r_term_deviation`):** Penalizes large control efforts and rapid input changes (slew rate) to smooth control actions. In this configuration, `r_term_control` (λ_2) is set to 0, disabling the direct penalty on control effort magnitude. This is a deliberate choice rather than an oversight. Constraining δ_t in absolute terms would penalize large but necessary scaling actions (e.g., in response to a sudden load spike) purely for their size, working against the responsiveness the controller is meant to provide. Instead, control smoothness is enforced solely through the slew-rate term `r_term_deviation` (λ), which penalizes the *change* be-

⁴<https://www.do-mpc.com/en/latest/index.html>

tween consecutive control actions rather than their magnitude. This allows the controller to take a large corrective step when the state is far from its target while still discouraging abrupt oscillation between steps.

- **Collocation parameters** (`collocation_deg`, `collocation_ni`): Define the internal solver discretization grid. Although intended for continuous-time models, `do-mpc` requires them as initialization parameters for discrete model execution.

Parameter	Symbol	Value	Description
<code>target_busy_time</code>	μ^*	0.8	Target TaskManager busy time
<code>target_idle_time</code>	ι^*	0.2	Target TaskManager idle time
<code>kappa</code>	κ	1.0	State transition control authority (Eq. 3.6–3.7)
<code>n_horizon</code> (event horizon)	T	3	MPC prediction lookahead horizon (steps)
<code>busy_cost_weight</code>	α_2	1.0	Weight for busy time tracking error
<code>idle_cost_weight</code>	α_1	1.0	Weight for idle time tracking error
<code>r_term_deviation</code>	λ_1	0.5	Slew rate penalty on control input change
<code>r_term_control</code>	λ_2	0.0	Weight on control input effort; kept at 0 so control magnitude is unconstrained, and smoothing is handled entirely by <code>r_term_deviation</code>
<code>deviation_lower_bound</code>	δ_{min}	−1.0	Minimum allowed scaling deviation term
<code>deviation_upper_bound</code>	δ_{max}	3.0	Maximum allowed scaling deviation term
<code>collocation_deg</code>	–	2	Orthogonal collocation polynomial degree
<code>collocation_ni</code>	–	2	Number of finite elements for collocation
<code>SYNC_PERIOD_SEC</code>	–	120 s	Control loop synchronisation interval
<code>BACKPRESSURE_THRESHOLD</code>	β_{thr}	0.1	Threshold (10%) triggering reactive scaling override
<code>utilization_factor</code>	–	0.8	Pod slot filling threshold

Table 6.3: Model predictive controller and autoscaling loop hyperparameters.

6.5 Results

To evaluate the effectiveness of the UpStream scaling algorithm against the baseline Kubernetes Horizontal Pod Autoscaler (HPA), comparative experiments across three distinct workload profiles were conducted: `TAXI_normalized`, `exponential_plateau`, and `static_10000`. The experiments were conducted with five runs per scaler and profile. Unless stated otherwise, the values reported below are means calculated over these runs, and the error bars denote the standard deviation across runs. It should be noted that the reported standard deviations captured two primary external factors that contribute to the run-to-run noise observed throughout this evaluation. First, the exact alignment between the start of the load profile and each scaler’s next reconciliation cycle is not controlled between runs. Since HPA and UpStream operate on different synchronization periods (see Section 6.4), a load ramp that begins immediately before a reconciliation tick is detected far sooner than one beginning immediately after, shifting the effective reaction delay by up to a full sync period. Second, as all experiments were executed on a shared network, co-located workloads cause noisy-neighbor effects on network resources, resulting in varying input rates. This is independent of the scaler under test. Both factors are difficult to fully control for given the experimental setup and should be kept in mind when interpreting the magnitude of differences reported below, particularly where they fall within one standard deviation.

Across some scenarios, UpStream demonstrates superior data processing performance and pipeline responsiveness, directly resolving the backpressure bottlenecks that severely limit HPA.

6.6 ETL Pipeline Evaluation

6.6.1 Throughput and Ingestion Performance

As shown in Figure 6.1, UpStream achieves significantly higher output processing rates compared to HPA. Under the `exponential_plateau` workload, UpStream reaches an average output throughput of 2,429 messages/second, compared to HPA’s 425 messages/second. This is a nearly 6× performance increase. Similar trends are observed for the `TAXI_normalized` (692 vs. 331 msg/s) and

`static_10000` (1,119 vs. 364 msg/s) workloads.

This difference in processing capability is equally evident in the database sink. In the MongoDB average inserts metric (Figure 6.2), UpStream reaches rates of 0.792-2.7 inserts/second. Conversely, HPA remains flatlined near 0.5-0.7 inserts/second on average.

Metric	TAXI_normalized		exponential_plateau		static_10000	
	UpStream	HPA	UpStream	HPA	UpStream	HPA
Kafka Output (msg/s)	692	331	2429	425	1119	364
MongoDB Output (insert/s)	0.792	0.512	2.7	0.596	1.11	0.675
Backpressure Fraction	0.0974	0.317	0.119	0.323	0.0906	0.316
Backpressure Fraction above target 10%	0.408	0.885	0.472	0.867	0.362	0.862
Busy Time Fraction	0.265	0.134	0.25	0.102	0.251	0.13
Idle Time Fraction	0.614	0.543	0.609	0.564	0.632	0.539
CPU utilization (%)	48.1%	60.2%	43.8%	51.8%	47.4%	55.6%
Memory utilization (%)	41.5%	65%	40.9%	63.2%	45%	63%
Provisioned Slots	7.06	4.24	14.4	5.6	8.3	4.78

Table 6.4: ETL: Performance and Metric Summary across Workload Profiles

6.6.2 Backpressure, Resource Provisioning, and Execution Dynamics

The standard HPA scaling mechanism relies on CPU and Memory utilization targets (set at 80%). As illustrated in Figures 6.3a and 6.3b, HPA maintains higher average CPU (51.8%–60.2%) and Memory (63%–65%) utilization per pod than UpStream (43.8%–48.1% CPU, 40.9%–45% Memory). This can be explained at the DAG level by the relationship between slot count and operator parallelism under Apache Flink’s Reactive Mode. HPA’s small slot pool (~ 4 -5) forces every operator to run with correspondingly low parallelism, so each active subtask must process a larger share of the partitioned stream to keep up with the offered load. Even where backpressure blocks some subtasks entirely, the subtasks that are actively processing carry a proportionally heavier load, driving up measured CPU utilization. UpStream’s larger slot pool (~ 7 -14) increases parallelism across the same DAG, spreading the identical total event volume across more subtask instances and reducing the CPU load this is consistent with UpStream showing both lower average utilization and substantially higher aggregate throughput (Table 6.4) Because these infrastructure targets are never breached for HPA, it fails to trigger scale-out events, capping provisioned task slots at approximately ~ 5 slots across all tested profiles. (Figure 6.3c).

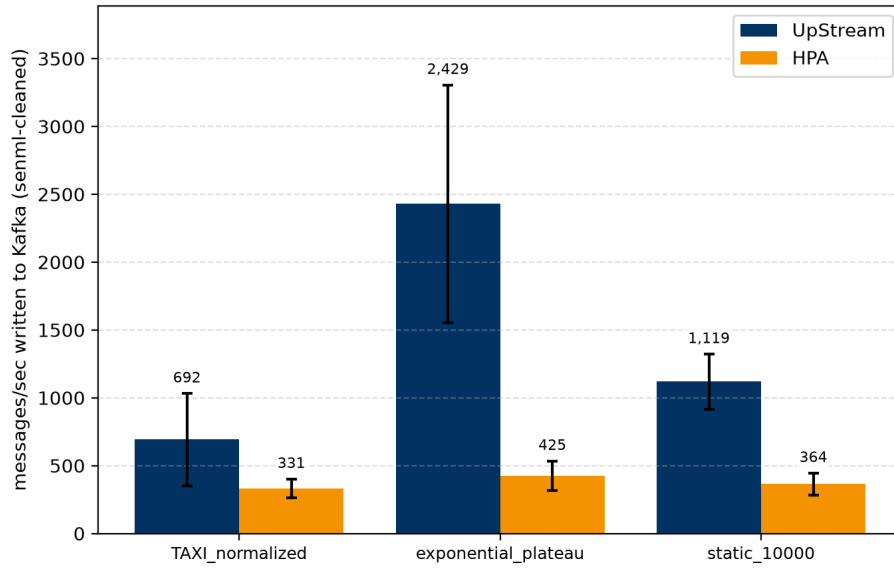


Figure 6.1: ETL Output Throughput: UpStream vs. HPA across workload profiles.

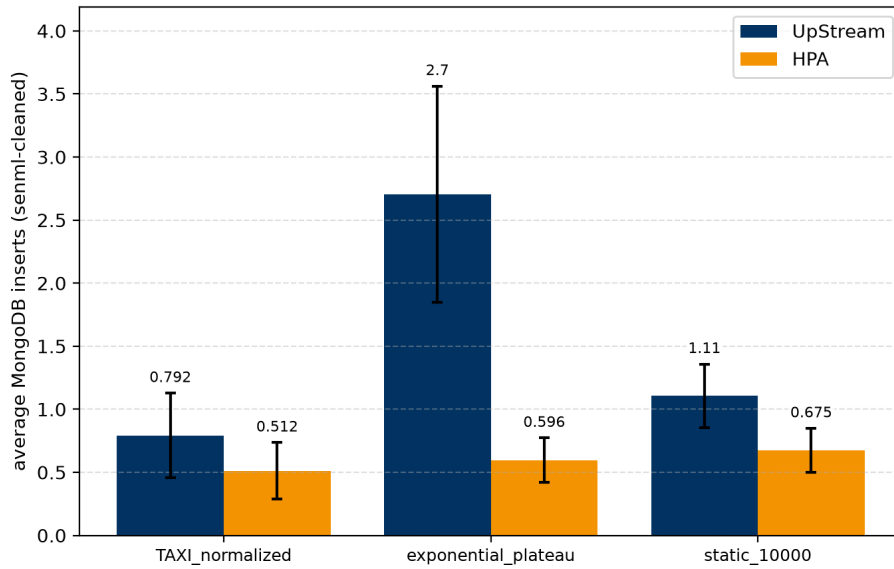


Figure 6.2: ETL MongoDB output: UpStream vs. HPA across workload profiles.

As a consequence of this under-provisioning, pipelines managed by HPA remain backpressured for 85% to 90% of their execution duration (Figure 6.3d).

In contrast, as can be seen in Figure 6.3c, UpStream provisions up to 4× the number of task slots during peak load cycles. This adaptive scaling reduces total time spent backpressured by approximately 35% (lowering backpressure fractions to 0.496-0.580) Figure 6.3d. By mitigating severe backpressure, UpStream redistributes blocked time into active processing, significantly increasing the mean busy time fraction (0.223-0.273 vs. HPA's 0.0795-0.120, Figure 6.3e) yet also maintaining a higher overall idle time fraction (0.544-0.580, Figure 6.3f).

6.6.3 Key Takeaways

The investigated load profiles all lead to a scenario where the deployed pipeline was in a backpressured state most of the time. Showing that backpressure is a strong indicator for under-provisioned pipelines, leading to great improvements when utilized as a scaling indicator.

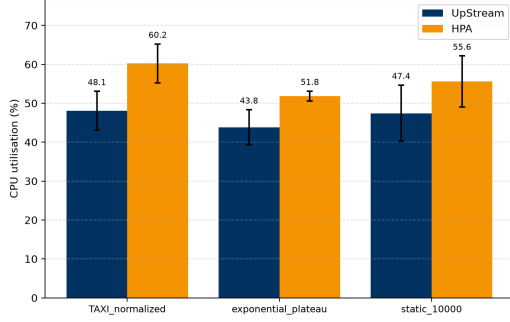
Through scaling out, UpStream requires a larger total resource footprint and incurs higher variability in allocated task slots. This directly translates into greatly improved throughput and target metric utilization. HPA's inability to detect streaming backpressure renders it unviable for high-volume ETL pipelines, whereas UpStream offers elasticity tailored for exactly these types of workloads.

6.7 STATS Pipeline Evaluation

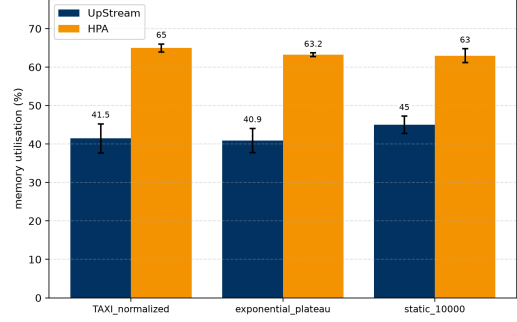
The second evaluated pipeline, STATS, computes windowed statistics over the incoming SenML stream and writes the results to MongoDB. In contrast to the ETL pipeline, STATS is a comparatively lightweight, non-blocking workload. In this setting, the results are less one-sided than for ETL: UpStream clearly improves the metrics it targets, but these improvements are roughly equivalent to those of HPA.

6.7.1 Throughput and Output Performance

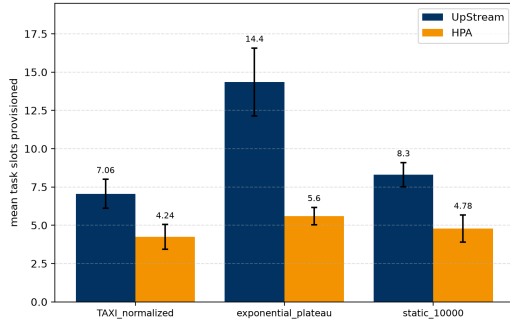
The average MongoDB insert rate is 1.1 (UpStream) vs. 1.08 (HPA) for `TAXI_normalized`, and 0.886 vs. 1.45 for `static_10000`. For the `TAXI_normalized`



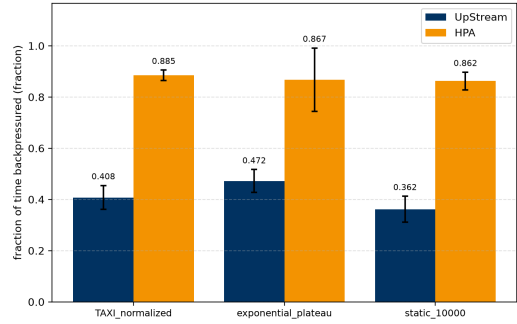
(a) Mean task manager CPU utilization (%) per load profile.



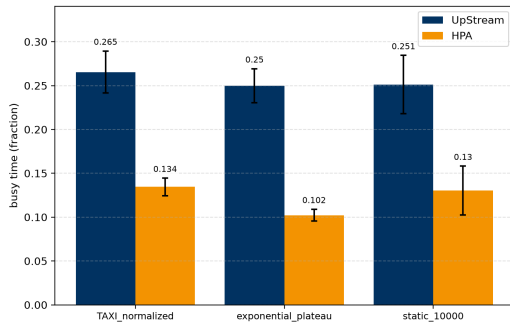
(b) Mean task manager memory utilisation (%) per load profile.



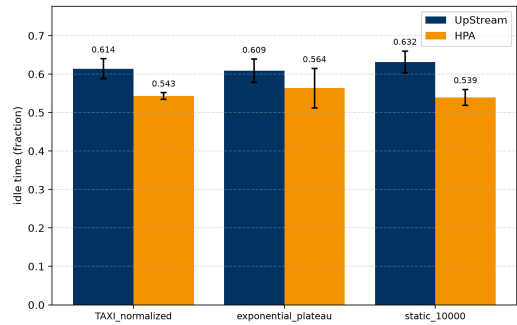
(c) Mean number of provisioned task slots per load profile.



(d) Fraction of time the backpressure exceeds the 10% target.



(e) Mean busy time fraction per load profile.



(f) Mean idle time fraction per load profile.

Figure 6.3: Comparison of UpStream and HPA for the ETL pipeline across load profiles: infrastructure utilization (CPU, memory), provisioned resources (task slots), operator load (busy and idle time), and backpressure. Bars show the mean over five runs, and error bars indicate ± 1 standard deviation across runs.

profile, the performance is almost identical, and the standard deviation is comparable. In the `static_10000` case, HPA performs significantly better, which can be explained by the fact that it, on average, provisions ~ 2 task slots more.

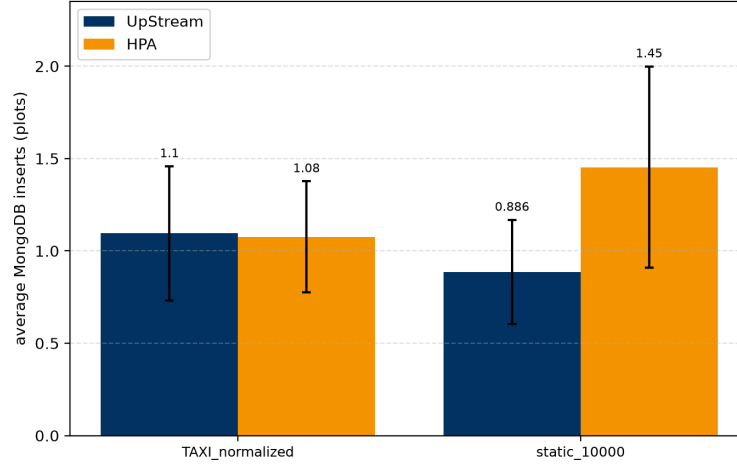


Figure 6.4: STATS MongoDB output: UpStream vs. HPA across workload profiles.

Metric	TAXI_normalized		static_10000	
	UpStream	HPA	UpStream	HPA
MongoDB Output (insert/s)	1.1	1.08	0.886	1.45
Backpressure Fraction	0.0744	0.127	0.0684	0.127
Backpressure Fraction above target 10%	0.337	0.775	0.3	0.787
Busy Time Fraction	0.274	0.165	0.268	0.139
Idle Time Fraction	0.636	0.694	0.653	0.722
CPU Utilization (%)	50.5%	56.5%	47.8%	46.9%
Memory Utilization (%)	59%	62.6%	60.1%	62%
Provisioned Slots	3.71	4.01	3.33	5.39

Table 6.5: STATS: Performance and Metric Summary across Workload Profiles

6.7.2 Target Metric Attainment

UpStream targets backpressure as well as busy and idle time, and Figures 6.5d, 6.5e, and 6.5f confirm that it moves both in the intended direction. The mean busy time fraction is 0.274 and 0.268 for UpStream, compared to 0.165 and 0.139 for HPA, an increase of about 66% and 92% for `TAXI_normalized` and `static_10000`, respectively. Idle time is reduced from 0.694 to 0.639 (-9%) and from 0.722 to

0.653 (−10%). Backpressure, which UpStream handles as its reactive gate, is lowered by roughly 83% ($0.127 \rightarrow 0.0774$) and 62% ($0.787 \rightarrow 0.3$), as shown in Figure 6.5d. Conversely, HPA maintains comparable or slightly higher resource utilization, averaging 56.5% and 46.9% CPU along with 62.6% and 62% memory across the load profiles.

6.7.3 Resource Provisioning and Utilization

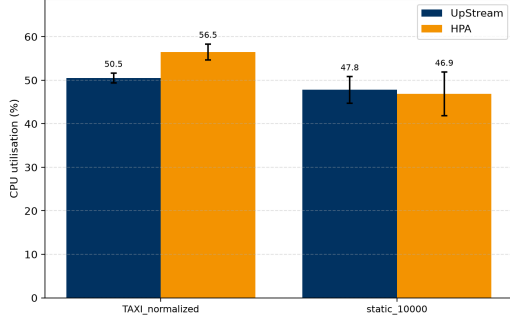
The resource provisioning dynamics depend on the workload (Figure 6.5c). Under `static_10000`, UpStream provisions on average 3.33 task slots compared to 5.39 for HPA (−38%). Under `TAXI_normalized`, the difference (3.71 vs. 4.01) lies within the standard deviation. Neither scaler comes close to exhausting its pods on average, and the differences in CPU and memory are small compared to the ETL evaluation.

6.7.4 Why Improved Target Metrics Do Not Translate into Better Performance

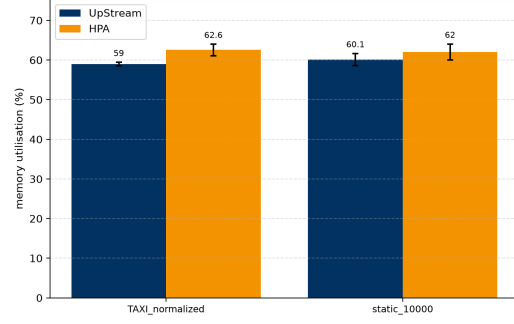
Although UpStream reaches its target metrics, this does not result in a measurable throughput advantage over HPA for STATS. The following reasons can play a part in this:

Backpressure is not significant enough. Under both profiles, HPA already keeps up with the offered load. Backpressure is very close to the target already. In ETL, HPA remained backpressured for 85–90% of the run, which left a large gap that UpStream could close. In STATS, HPA is still backpressured roughly 78% of the time, but the fraction is already very close to the target threshold of Upstream (0.127). The signal UpStream exploits therefore has little headroom to improve, and any gain in busy time or idle time is comparable to CPU and Memory targets.

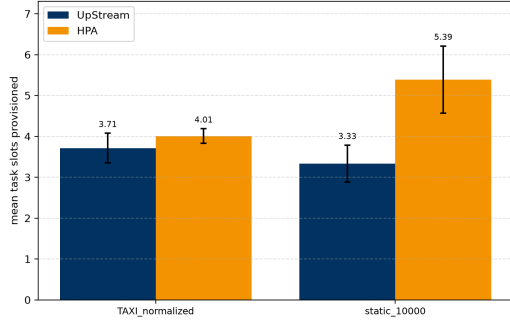
Delayed reaction during load ramp. Both scalers start from the same two-task-slot deployment, yet HPA issues its first scale-out after ≈ 60 s (Figure 6.6) and ≈ 75 s (Figure 6.7), whereas UpStream first scales after ≈ 195 s and ≈ 120 s, respectively. In this window, UpStream’s average task-manager CPU rises to



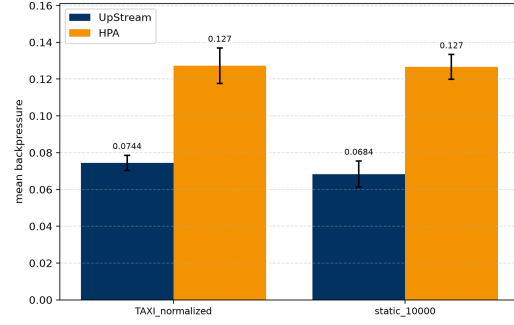
(a) Mean task manager CPU utilization (%) per load profile.



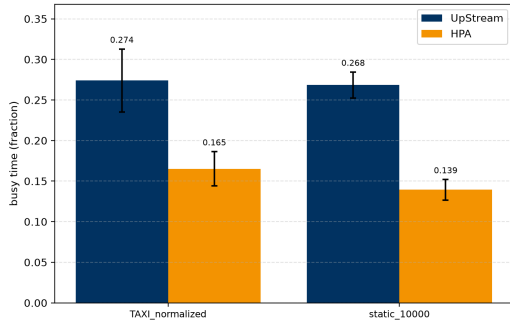
(b) Mean task manager memory utilization (%) per load profile.



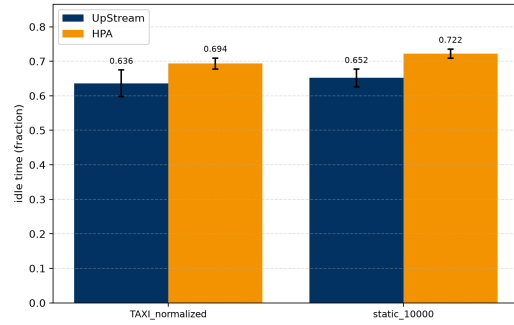
(c) Mean number of provisioned task slots per load profile.



(d) Mean backpressure fraction per load profile.



(e) Mean busy time fraction per load profile.



(f) Mean idle time fraction per load profile.

Figure 6.5: Comparison of UpStream and HPA for the STATS pipeline across load profiles: infrastructure utilization (CPU, memory), provisioned resources (task slots), operator load (busy and idle time), and backpressure. Bars show the mean over five runs, and error bars indicate ± 1 standard deviation across runs.

$\approx 90\text{--}95\%$, i.e., the unmitigated response of the pipeline to the ramp, while HPA’s curve reflects the response *after* its own scale-out, which dilutes the average. UpStream’s target signals lag behind this resource signal. Backpressure only becomes higher once the pipeline saturates ($\approx 70\text{--}80\text{ s}$) and reaches ≈ 0.18 backpressure shortly before the first scale-out. On top of this signal delay, UpStream’s sync period as mentioned in section 6.4 (120 s) adds up to the detection latency, compared to HPA’s default 15 s synchronization period.

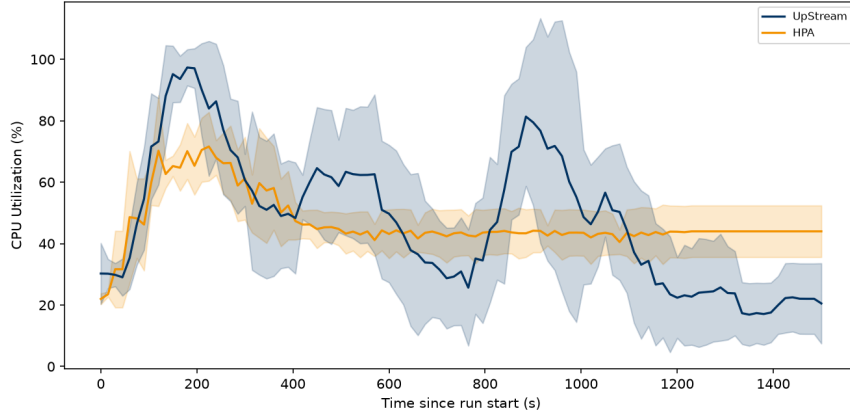
Oscillating allocations. After the initial phase, HPA converges to a constant configuration (≈ 5.5 slots under `static_10000`, and 4 slots under `TAXI_normalized`). UpStream’s allocation, in contrast, keeps changing (between about 3 and 5.5 slots for both profiles). Similar oscillation behaviour is also observed in CPU and backpressure for Figures 6.6 and 6.7. Looking at figure 6.7c, this oscillation is what allows Upstream to compensate for the initial missed load spike in the case of `TAXI_normalized`, leading to a similar output performance. For `static_10000` (Figure 6.6c), UpStream is never able to compensate.

Resource metrics do not reveal a bottleneck that HPA misses. For ETL, HPA’s CPU and memory targets were never breached, even though the pipeline was starved. In STATS, however, HPA does scale out, and it does so early. Its average CPU utilization stays comparable to UpStream. One of the motivations for UpStream is that it is an infrastructure metric that stays silent while the pipeline is blocked; thus, it does not occur here, and the two scalers reach comparable results.

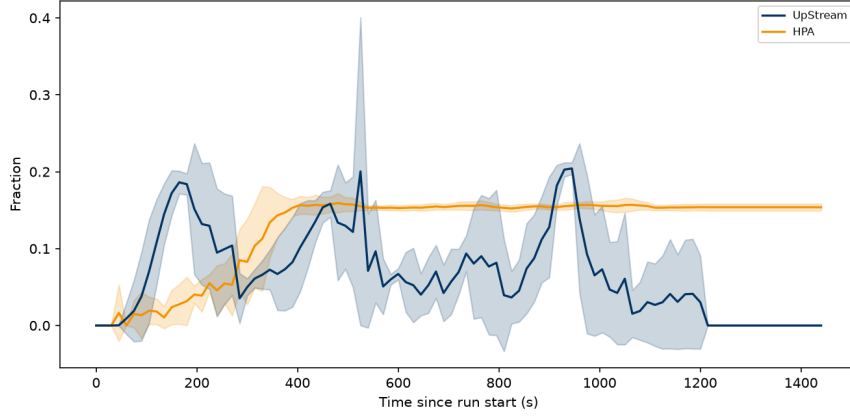
Measurement noise limits the conclusions. With $n = 5$ runs and a large standard deviation, several of the differences in Table 6.5 are overlapping. Moreover, the difference in sync period between the two scalers, as well as timing differences in the launching of the tests, contribute to the noise.

6.7.5 Key Takeaways

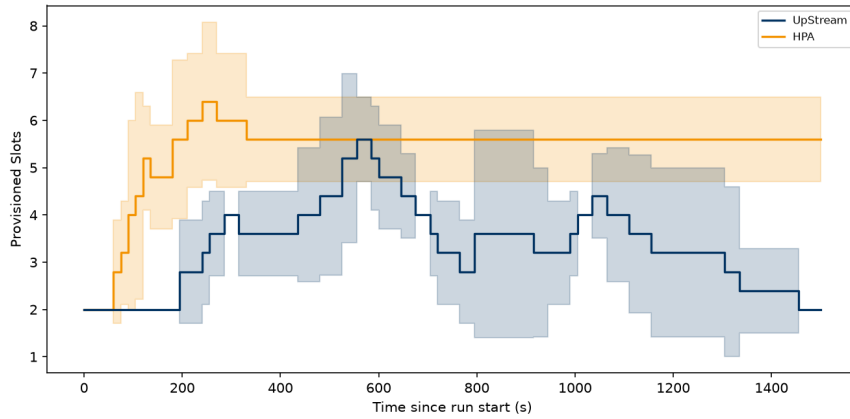
On STATS, UpStream reliably improves the metrics it regulates, but the Improvement does not translate into a consistent advantage over HPA.



(a) Average task manager CPU utilization over time for the STATS pipeline under the `static_10000` workload, comparing HPA and UpStream.

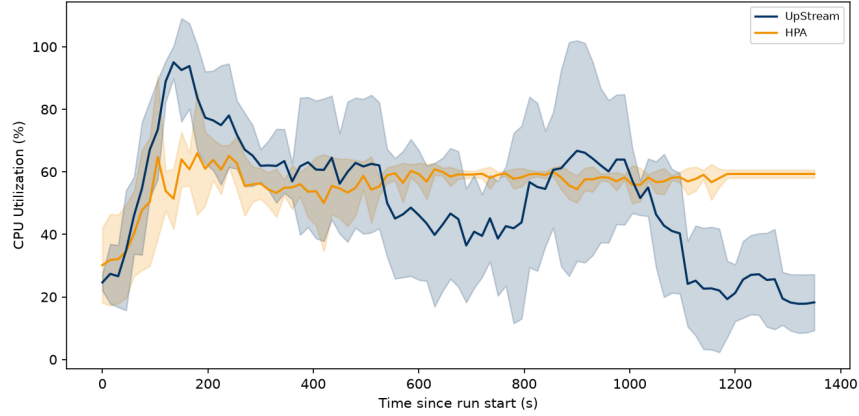


(b) Backpressure fraction over time for the STATS pipeline under the `static_10000` workload, comparing HPA and UpStream.

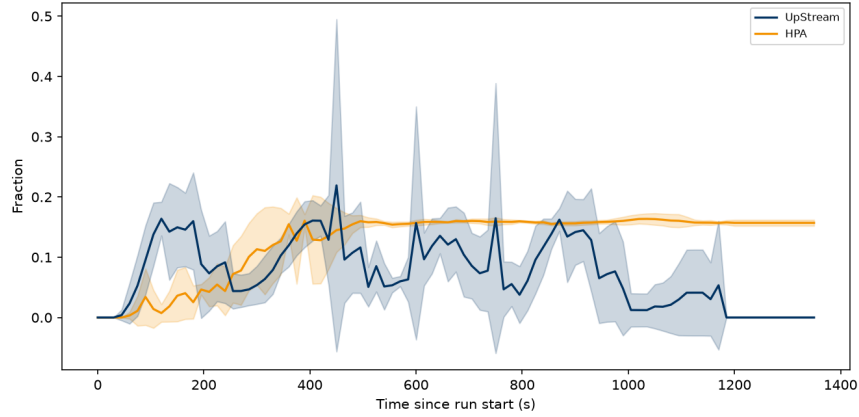


(c) Provisioned task slots over time for the STATS pipeline under the `static_10000` workload, comparing HPA and UpStream.

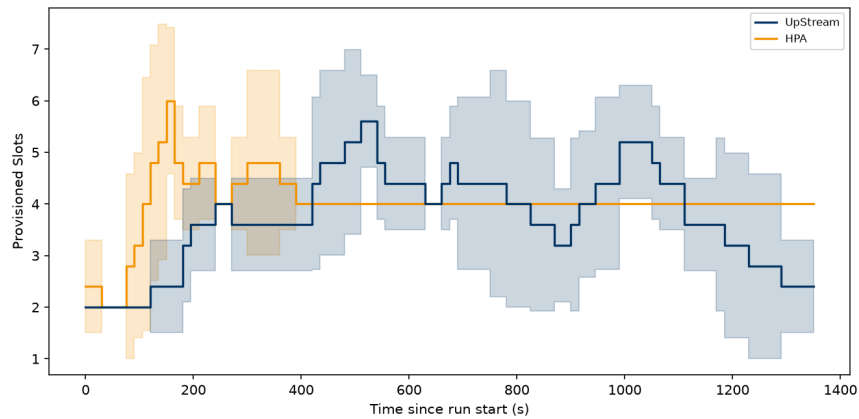
Figure 6.6: Resource utilization, backpressure, and provisioned task slots over time for the STATS pipeline under the `static_10000` workload. Each line shows the mean over five runs, and the shaded band indicates ± 1 standard deviation across runs.



(a) Average task manager CPU utilization over time for the STATS pipeline under the `TAXI_normalized` workload, comparing HPA and UpStream.



(b) Backpressure fraction over time for the STATS pipeline under the `TAXI_normalized` workload, comparing HPA and UpStream.



(c) Provisioned task slots over time for the STATS pipeline under the `TAXI_normalized` workload, comparing HPA and UpStream.

Figure 6.7: Resource utilization, backpressure, and provisioned task slots over time for the STATS pipeline under the `TAXI_normalized` workload. Each line shows the mean over five runs, and the shaded band indicates ± 1 standard deviation across runs.

- **Target metrics are met.** UpStream increases the busy time fraction by 66% and 93%, lowers the idle time fraction by 8% and 10%, and reduces the mean backpressure fraction by 41% and 46% for `TAXI_normalized` and `static_10000`, respectively. The time spent above the 10% backpressure threshold drops from 0.775 to 0.337 and from 0.787 to 0.300.
- **Output performance is not improved.** The MongoDB insert rate is equal under `TAXI_normalized` and lower for UpStream under `static_10000`, where HPA provisions significantly more task slots on average.
- **Reactivity and stability are the price.** UpStream's first scaling decision occurs after HPAs. This is explained by its 120s sync period. Its allocation keeps oscillating, whereas HPA converges to a constant configuration.

The reason is that STATS does not have a bottleneck that infrastructure metrics fail to see: HPA already scales out on CPU, and its backpressure is low compared to ETL, which leaves little headroom for the backpressure gate. Part of UpStream's benefit is therefore not applicable.

6.8 PRED Pipeline Evaluation

The third evaluated pipeline, PRED, consumes the cleaned SenML stream and is evaluated under the same two workload profiles as STATS. For PRED, no output metrics were recorded. The evaluation, therefore, cannot compare processing performance directly. Instead, it analyses whether the pipeline showed signs of insufficient capacity, how many resources each scaler provisions, and how the resulting allocation and utilization evolve over time. In contrast to ETL and STATS, UpStream never scales the number of task slots under either of the two load profiles, while HPA continuously adapts the number of task slots.

6.8.1 Capacity Based on Backpressure

Under both profiles and for both scalers, backpressure is practically absent (Table 6.6, Figure 6.10d). The mean backpressure fraction is at most 0.00113, and the time spent above the 10% backpressure threshold is 0 in all cases except HPA

under `TAXI_normalized` (0.46%). This is due to a large spike in one of the experiment runs, which leads to such a large error. The remaining differences are within the run-to-run variation and cannot be interpreted as an effect.

Since no output metrics exist, the absence of backpressure is an indicator, but not proof, that both configurations kept up with the load (see Section 6.8.6).

Metric	TAXI_normalized		static_10000	
	UpStream	HPA	UpStream	HPA
Backpressure Fraction	0.00015	0.00113	0.000372	0.0000622
Backpressure Fraction above target 10%	0	0.00455	0	0
Busy Time Fraction	0.407	0.530	0.327	0.442
Idle Time Fraction	0.577	0.434	0.664	0.519
CPU Utilization (%)	75.2%	45.0%	71.3%	45.5%
Memory Utilization (%)	92.9%	43.0%	87.4%	50.1%
Provisioned Slots	2.00	4.28	2.00	4.08

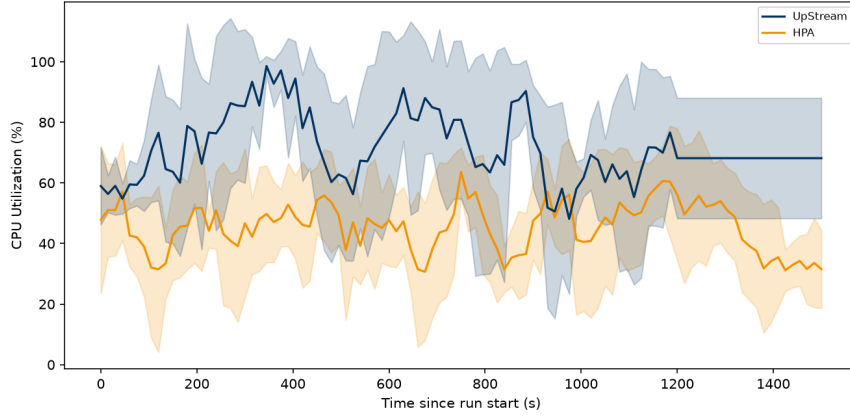
Table 6.6: PRED: Metric Summary across Workload Profiles

6.8.2 Resource Provisioning

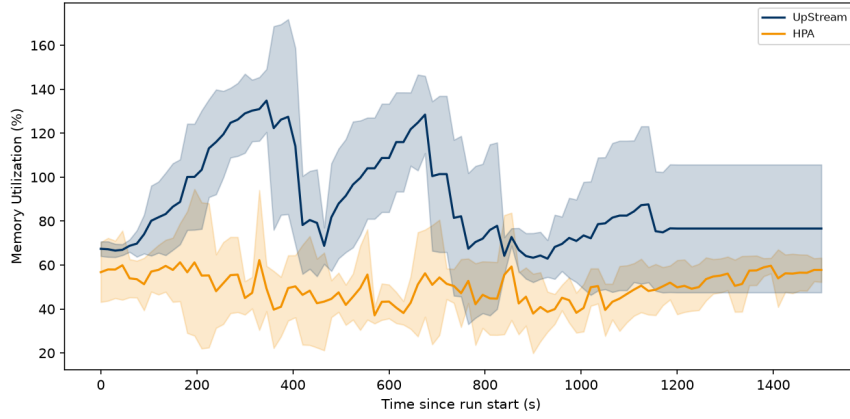
The provisioned resources differ clearly (Figure 6.10c). UpStream never leaves its initial configuration of one replica with two task slots: the allocation is constant over the whole run in all UpStream runs. HPA provisions on average 4.28 (`TAXI_normalized`) and 4.08 (`static_10000`) task slots, roughly twice as many, i.e., UpStream uses $\approx 50\%$ fewer slots. As backpressure never occurs for PRED, no scale-out is triggered, busy and idle time do reach short spikes above the target thresholds in section 6.4, but these are short-lived and not detected by UpStream.

6.8.3 Resource utilization and Short-Term Resource Spikes

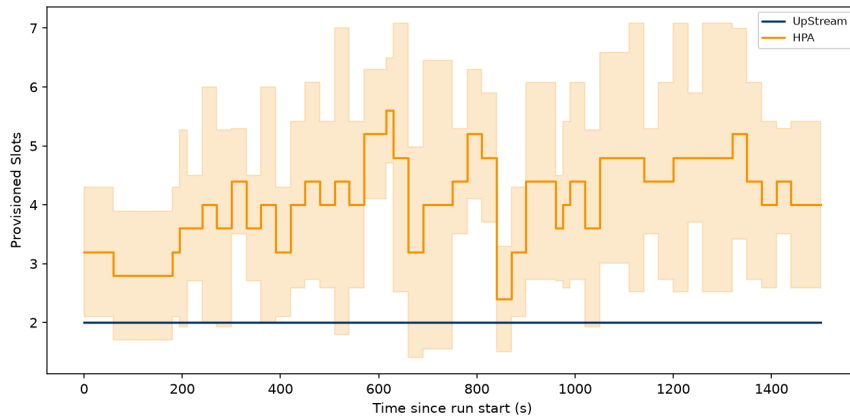
The lower allocation results in a considerably higher utilization of the single UpStream pod (Figures 6.10a and 6.10b). The mean CPU utilization is 75.2% and 71.3% compared to 45.0% and 45.5% for HPA, and the mean memory utilization is 92.9% and 87.4% compared to 43.0% and 50.1%. Both averages are close to or above the 80% target that HPA is configured with. The time series show that this is more than a shifted mean (Figures 6.8 and 6.9): the group-mean CPU of UpStream repeatedly reaches $\approx 95\text{--}100\%$, and the memory utilization reaches $\approx 130\%$ during the execution. Because the utilization is reported relative to the



(a) Average task manager CPU utilization over time for the PRED pipeline under the **static_10000** workload, comparing HPA and UpStream.

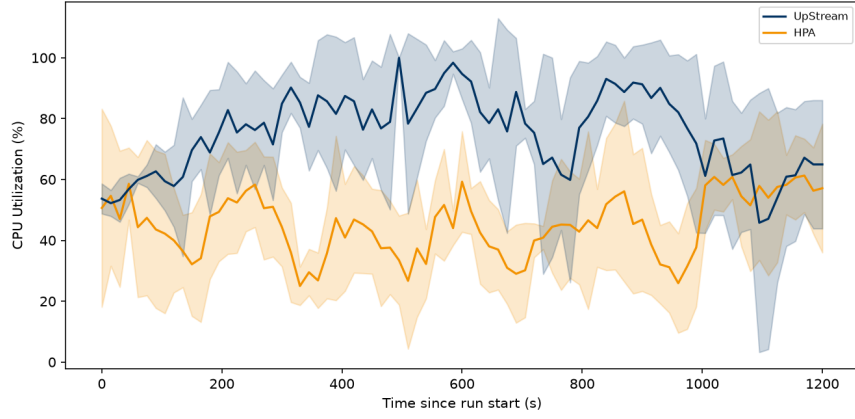


(b) Average task manager memory utilization over time for the PRED pipeline under the **static_10000** workload, comparing HPA and UpStream.

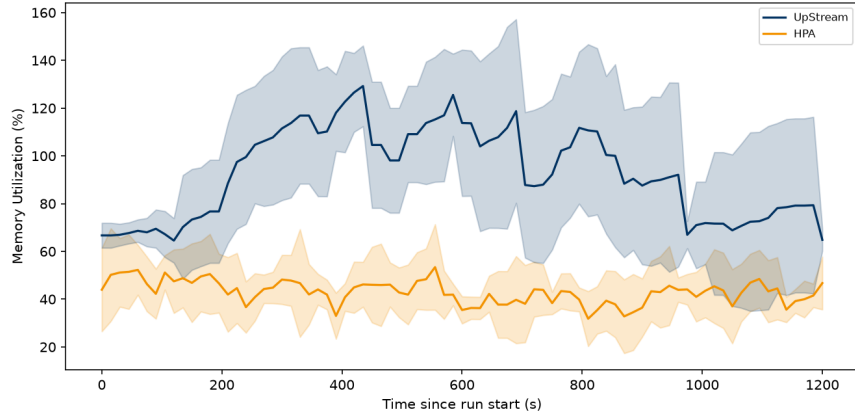


(c) Provisioned task slots over time for the PRED pipeline under the **static_10000** workload, comparing HPA and UpStream.

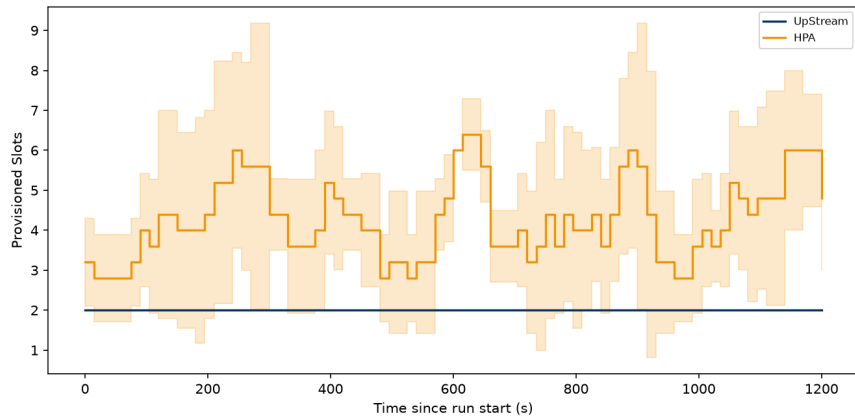
Figure 6.8: Resource utilization (CPU, memory) and provisioned task slots over time for the PRED pipeline under the **static_10000** workload. Each line shows the mean over five runs, and the shaded band indicates ± 1 standard deviation across runs.



(a) Average task manager CPU utilization over time for the PRED pipeline under the `TAXI_normalized` workload, comparing HPA and UpStream.



(b) Average task manager memory utilization over time for the PRED pipeline under the `TAXI_normalized` workload, comparing HPA and UpStream.



(c) Provisioned task slots over time for the PRED pipeline under the `TAXI_normalized` workload, comparing HPA and UpStream.

Figure 6.9: Resource utilization (CPU, memory) and provisioned task slots over time for the PRED pipeline under the `TAXI_normalized` workload. Each line shows the mean over five runs, and the shaded band indicates ± 1 standard deviation across runs.

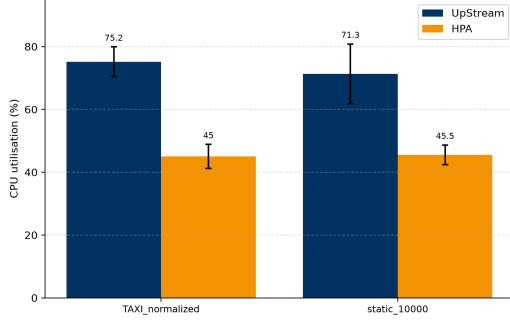
requested resources, values above 100% mean that the pod temporarily uses more than it requested. This is permitted by Kubernetes up to the configured limits. For the small task manager deployment from table 6.2, this is managed by the Apache Flink Helm chart. It configures its deployed pods without resource limits and therefore allows these temporary spikes. For the additional deployments of UpStream, limits = requests. Due to this configuration, Upstream, whose decisions do not depend on infrastructure metrics, tolerates these short-term spikes above the request. HPA, in contrast, reacts to exactly these signals: with a memory utilization of $\approx 130\%$ and a target of 80%, a single pod already triggers a scaling decision. This explains why HPA holds about twice as many slots, and why its resulting per-pod utilization ($\approx 45\%$) stays far below the target.

6.8.4 Scaling Dynamics

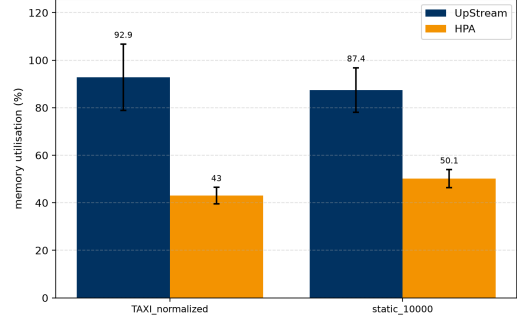
The time series (Figures 6.8 and 6.9) show a contrast to STATS. Whereas in STATS, HPA converged to a constant configuration and UpStream oscillated, the roles are reversed for PRED. The HPA allocation keeps changing over the entire run (the group mean varies between ≈ 2.5 and ≈ 6 task slots), whereas the UpStream allocation is constant. A likely cause for this oscillation of HPA is the feedback loop of a utilization-based scaler: adding a replica lowers the average utilization per pod, which later allows the scaler to remove it again, after which the utilization rises above the target once more. This is an effect in addition to the observed spikes in the load profile of PRED.

6.8.5 Busy and Idle Time

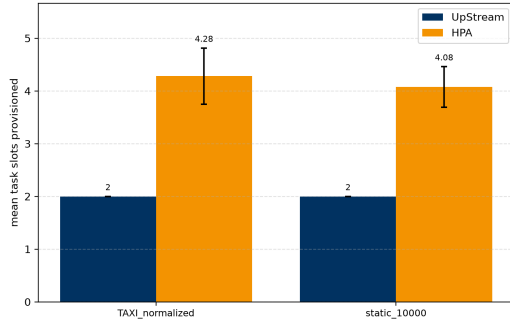
Contrary to STATS, UpStream does not achieve a higher busy time than HPA for PRED: the busy time fraction is 23% and 26% lower (0.407 vs. 0.530 and 0.327 vs. 0.442), and the idle time fraction is 33% and 28% higher (Figures 6.10e and 6.10f). This does not indicate that UpStream regulates worse. With backpressure at zero and a moderately busy time, its controller has no reason to scale out, and since the allocation is already at its minimum, it cannot scale in either. Its target metrics are therefore not regulated but merely observed. Both busy and idle time fluctuate strongly between 0.1 and 1.0 in the time series for both scalers.



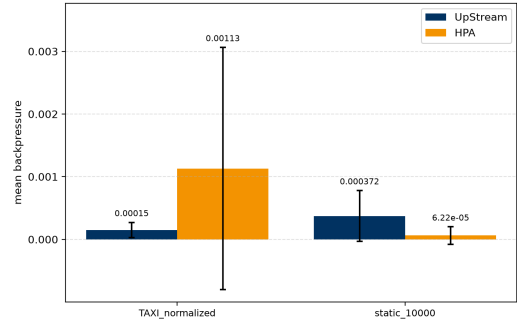
(a) Mean task manager CPU utilization (%) per load profile.



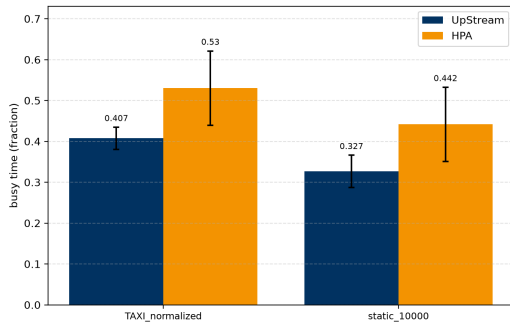
(b) Mean task manager memory utilization (%) per load profile.



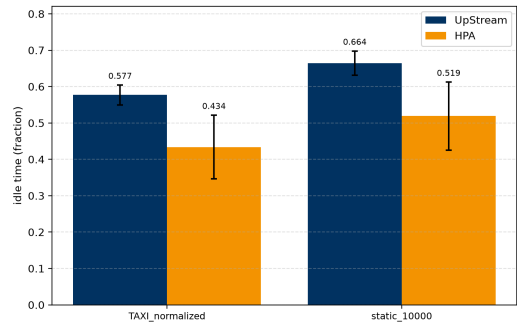
(c) Mean number of provisioned task slots per load profile.



(d) Mean backpressure fraction per load profile.



(e) Mean busy time fraction per load profile.



(f) Mean idle time fraction per load profile.

Figure 6.10: Comparison of UpStream and HPA for the PRED pipeline across load profiles: infrastructure utilization (CPU, memory), provisioned resources (task slots), operator load (busy and idle time), and backpressure. Bars show the mean over five runs, and error bars indicate ± 1 standard deviation across runs.

6.8.6 Limitations

Assessing if UpStream’s single pod processed the same volume as HPA’s larger allocation remains future work. Comparing output performance is of particular future interest because the recorded CPU utilization of UpStream is near saturation, and its memory exceeds the request during peaks. The observed absence of backpressure supports, but does not prove, that no capacity shortfall occurred. In addition, the results are based on $n = 5$ runs with substantial variance, and the curves for the input load of the two groups are offset by a few seconds, so timing differences of this size should not be interpreted.

6.8.7 Key Takeaways

- **UpStream never scales for PRED.** As backpressure does not occur and the other target metrics are never detected to be above the target threshold, UpStream keeps its initial configuration of two task slots in all runs, whereas HPA provisions 4.08–4.28 slots on average. This corresponds to about 51–53% fewer provisioned resources.
- **The default Flink pod configuration allows for overutilization.** The single UpStream pod runs at 71–75% CPU and 87–93% memory on average, with short-term peaks of $\approx 100\%$ CPU and $\approx 130\%$ of the requested memory. UpStream tolerates these spikes, since it does not use infrastructure metrics, while the same values trigger HPA.
- **HPA oscillates, UpStream does not.** HPA continuously adds and removes replicas, each of which implies a rescaling of the job, and it remains at a low per-pod utilization ($\approx 45\%$). In STATS, the roles were reversed, which shows that oscillation depends on the interaction between the scaler and the pipeline rather than on the scaler alone.
- **No claim on throughput is possible.** Without output metrics, the results show that UpStream avoids visible backpressure with half the resources, but not that its output equals that of HPA.

7 Conclusion

The resource signals available to a generic autoscaler, CPU and memory utilization, are only indirect indicators of whether a stream processing pipeline keeps up with its input. This thesis introduced UpStream, a hybrid reactive/predictive autoscaler for Apache Flink on Kubernetes that uses metrics of the DSP engine itself, and evaluated it against the standard Kubernetes Horizontal Pod Autoscaler (HPA) on a bare-metal testbed. This chapter summarizes the results, discusses under which conditions UpStream is beneficial, states the limitations of the evaluation, and outlines future work.

7.1 Summary

UpStream controls the size of the TaskManager pool from outside the DSP engine, while Apache Flink’s Reactive Mode adapts the operator parallelism to the available task slots. The external controller combines two mechanisms: a reactive gate that scales the deployment in proportion to the observed backpressure and a Model Predictive Controller (MPC) that drives busy and idle time towards target values whenever backpressure is negligible. A greedy allocation step then maps the resulting number of task slots to TaskManager pods of three different sizes. We evaluated UpStream against an HPA that scales on CPU and memory utilization (80% target, tuned to 15-second reaction windows). The evaluation used the ETL, STATS, and PRED pipelines of RIoT Bench, ported to Apache Flink through Apache Beam, and load profiles derived from a constant load, an exponential ramp, and a real TAXI trace. All results are based on five runs per scaler and profile.

- **ETL:** Under HPA, the pipeline was backpressured for 86–89% of the time, since its CPU and memory targets were never breached and it stayed at about five task slots. UpStream reacted to this backpressure and increased

the output throughput by a factor of 2.1 to 5.7 (692 vs. 331, 2,429 vs. 425, and 1,119 vs. 364 messages/second for the three profiles). The time spent above 10% backpressure dropped to 36–47%. UpStream provisioned on average 1.7 to 2.6 times as many task slots for this.

- **STATS:** STATS is a lighter workload, and HPA already scaled out on CPU utilization and was backpressured only moderately. UpStream increased the busy time fraction by 66% and 93% and reduced the mean backpressure by 41% and 46%. However, this did not improve the output: the MongoDB insert rate was equal under `TAXI_normalized` (1.10 vs. 1.08 inserts/second) and lower under `static_10000` (0.886 vs. 1.45), where UpStream used 38% fewer task slots (3.33 vs. 5.39). UpStream’s first scale-out came about 45 to 135s after that of HPA.
- **PRED:** Backpressure was practically absent, so UpStream never left its initial configuration of two task slots, while HPA held 4.1–4.3 slots on average in response to the utilization of the single pod. The UpStream pod ran at 71–75% CPU and 87–93% memory on average, with peaks of about 130% the requested memory. As no output metrics were recorded for PRED, we cannot state whether the throughput of both scalers was equal.

7.2 Discussion

The benefit of UpStream depends on the bottleneck. The results support the central hypothesis of this thesis only in part. Where a pipeline is starved in a way that CPU and memory do not reveal, as in ETL, reacting to backpressure leads to large improvements, and the ETL results show that infrastructure metrics alone can leave a pipeline backpressured for most of its runtime. Where the infrastructure metrics correlate well with the load, as in STATS, HPA reaches comparable output, and the advantage of UpStream is limited to improved target metrics and, under a constant load, a lower resource footprint. The value of a DSP-specific signal, therefore, depends on how visible the bottleneck already is to the generic signals.

Reactivity and stability are a trade-off. Because a rescaling of the Apache Flink job takes about 120s, the sync period of UpStream was set to the same value, while HPA evaluates its metrics every 15s. Together with signals that only become visible once the pipeline saturates, this delayed the first reaction of UpStream to a load ramp, and the pipeline was exposed to CPU peaks of 90–95% during that time. The expected smoothing effect of the MPC was not observed consistently either: UpStream’s allocation oscillated in STATS, whereas in PRED it was HPA that continuously added and removed replicas. Oscillation, therefore, appears to be a property of the interaction between scaler and pipeline rather than of one scaler alone.

Signals can disagree. PRED shows that DSP-specific and infrastructure metrics can point in opposite directions: the memory and CPU utilization of the single pod would have triggered a scale-out under HPA, while backpressure and busy time gave UpStream no reason to act. Without output metrics, we cannot decide which signal was right. This suggests that neither signal family suffices on its own.

7.3 Limitations

The results have to be interpreted with the following limitations in mind.

- **Pod allocation rounding inconsistency.** As noted in Section 3.7, the pod allocation algorithm (Algorithm 3) contains a discrepancy between the *utilization_factor*-scaled threshold used in its loop condition and the unscaled node capacity used in its update step. A corrected version replaces both with the same scaled quantity,

$$\text{effective_capacity} = \text{round}(\text{task_slot_capacity} \times \text{utilization_factor}), \quad (7.1)$$

used consistently for both the loop condition and the subtraction. Since all results reported in this thesis were produced with *utilization_factor* = 0.8 and the incorrect algorithm, individual allocation decisions deviate from what this corrected formulation would have produced. However, the per-decision inputs to the allocator (the task-slot demand and node configu-

ration at each allocation call) were not logged during the evaluation, so the extent of this deviation cannot be reconstructed. Quantifying its impact would require running the evaluation again with the corrected algorithm. This discrepancy is confined entirely to the node-allocation step. (Section 3.7) and does not affect the reactive scaling gate or the MPC-based predictive component (Sections 3.5, 3.6), which determine task-slot demand independently of how that demand is subsequently mapped onto nodes. Consequently, the general throughput and scaling-dynamics results in Chapter 6 are unaffected, but the reported resource provisioning figures should be interpreted with this caveat in mind. Future work replicating this evaluation should log per-call allocator inputs to allow such discrepancies to be audited directly.

- **Baseline.** The HPA scales only on CPU and memory. The advantage in ETL, therefore, demonstrates the value of a backpressure signal over generic utilization metrics, not necessarily over other DSP-aware autoscalers, such as the autoscaler of the Apache Flink Kubernetes Operator or an HPA on custom metrics like consumer lag or latency.
- **No isolation of components.** UpStream was evaluated as a whole. The contributions of the MPC, the backpressure gate, and the multi-size pod allocation were not evaluated separately. In ETL, backpressure exceeds the gate threshold most of the time, so the gate probably dominates the decisions there, but we did not analyze this.
- **Cost in task slots.** Resource cost was compared in provisioned task slots. As medium and large pods reserve twice the CPU and memory per slot compared to small pods (Table 6.2), slot counts only approximate the cost of UpStream when it uses larger pods.
- **No checkpointing.** Reactive Mode had to be used without checkpointing, so every rescaling discards operator state and in-flight events. This applies to both scalers, but the amount of lost data depends on the number of rescalings, which differs between the scalers and may influence the output metrics.
- **Statistical power and scope.** Each configuration was run five times,

with runs lasting about 15–25 minutes, i.e., only a few control cycles at a sync period of 120s. Several differences lie within one standard deviation. No output metrics exist for PRED and all experiments were conducted on a single three-node testbed.

7.4 Future Work

- **Actual workload prediction.** The current MPC model has no autonomous dynamics, so it regulates the current state and predicts the impact of control dynamics on the system but no system behavior outside of the control inputs. Adding a forecast of the input rate would make the controller predictive in a stricter sense.
- **Additional signals.** Combining backpressure and busy time with input rate, end-to-end latency, and utilization metrics could resolve cases such as PRED, where the signals disagree.
- **Stronger baselines and isolation.** A comparison with the Apache Flink Kubernetes Operator autoscaler and an HPA targeting domain-specific metrics, as well as isolation of the reactive gate, the MPC, and the pod allocation, would show which part of UpStream is responsible for which effect.
- **Cost and latency.** Reporting cost in CPU cores, memory, and pod-seconds, and adding end-to-end latency, would allow a more complete assessment, including the effect of larger pods on network overhead.
- **Fault-tolerant rescaling.** Resolving the interaction between Reactive Mode, checkpointing, and Kafka sources would allow evaluating UpStream without data loss during rescaling.
- **Alternative DSP Engines.** Investigating alternative stream processing engines such as Apache Spark [41] or even Kubernetes native frameworks such as Numaflow¹ might provide a more in-depth view on stream processing autoscaling.

¹<https://numaflow.numaproj.io/>

In summary, UpStream is a useful autoscaler when a pipeline is bottlenecked in a way that generic resource metrics do not reveal, but on lightweight pipelines, it currently offers no measurable throughput advantage over a well-tuned HPA, and its slower reaction and its allocation stability leave room for improvement.

Bibliography

- [1] Shaimaa Safaa Ahmed Alwaisi, Maan Nawaf Abboud, Luma Fayege Jalil, Shahreen Kasim, Mohd Farhan Mohd Fudzee, Ronal Hadi, and Mohd Arfian Ismail. A review on big data stream processing applications: Contributions, benefits, and limitations. *JOIV: International Journal on Informatics Visualization*, 5(4):456–460, 2021.
- [2] Henrique CM Andrade, Buğra Gedik, and Deepak S Turaga. *Fundamentals of stream processing: application design, systems, and analytics*. Cambridge University Press, 2014.
- [3] Garcia Rafael Banos, Oresti and Alejandro Saez. MHEALTH. UCI Machine Learning Repository, 2014. DOI: <https://doi.org/10.24432/C5TW22>.
- [4] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM*, 13(7):422–426, jul 1970.
- [5] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4), 2015.
- [6] Valeria Cardellini, Francesco Lo Presti, Matteo Nardelli, and Gabriele Russo Russo. Runtime adaptation of data stream processing systems: The state of the art. *ACM Comput. Surv.*, 54(11s), September 2022.
- [7] Marcos Dias de Assuncao, Alexandre da Silva Veith, and Rajkumar Buyya. Distributed data stream processing and edge computing: A survey on resource elasticity and future directions. *Journal of Network and Computer Applications*, 103:1–17, 2018.

- [8] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Commun. ACM*, 51(1):107â113, January 2008.
- [9] Brian Donovan and Dan Work. New york city taxi trip data (2010-2013), 2016.
- [10] Felix Fiedler, Benjamin Karg, Lukas LÃ¼ken, Dean Brandner, Moritz Heinlein, Felix Brabender, and Sergio Lucia. do-mpc: Towards fair nonlinear and robust model predictive control. *Control Engineering Practice*, 140:105676, 2023.
- [11] Gabriel Rustick Fim, Gabriele Mencagli, and Dalvan Griebler. Benchmarking batch and stream processing execution modes in apache flink. *Computing*, 108(1), January 2026.
- [12] Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. A survey on the evolution of stream processing systems. *The VLDB Journal*, 33(2):507–541, November 2023.
- [13] Martin Hirzel, Robert Soulé, Scott Schneider, Buğra Gedik, and Robert Grimm. A catalog of stream processing optimizations. *ACM Comput. Surv.*, 46(4), March 2014.
- [14] Fabian Hueske and Vasiliki Kalavri. *Stream processing with Apache Flink: fundamentals, implementation, and operation of streaming applications*. O’Reilly Media, 2019.
- [15] IANA. Sensor Measurement Lists (SenML) Parameters. IANA registry.
- [16] Shigeru Imai, Stacy Patterson, and Carlos A. Varela. Uncertainty-aware elastic virtual machine scheduling for stream processing systems. In *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 62–71, 2018.
- [17] Emilio Incerto, Mirco Tribastone, and Catia Trubiani. Combined vertical and horizontal autoscaling through model predictive control. In *Euro-Par 2018: Parallel Processing: 24th International Conference on Parallel and Distributed Computing, Turin, Italy, August 27-31, 2018, Proceedings 24*, pages 147–159. Springer, 2018.

- [18] Cullen Jennings, Zach Shelby, Jari Arkko, Ari Kernén, and Carsten Bormann. Sensor Measurement Lists (SenML). RFC 8428, IETF, 2018.
- [19] Nisarg S Joshi, Raghav Raghuwanshi, Yash M Agarwal, B Annappa, and DN Sachin. Arima-pid: container auto scaling based on predictive analysis and control theory. *Multimedia Tools and Applications*, 83(9):26369–26386, 2024.
- [20] Li Ju, Prashant Singh, and Salman Toor. Proactive autoscaling for edge computing systems with kubernetes. *CoRR*, abs/2112.10127, 2021.
- [21] Martin Junghanns, André Petermann, Niklas Teichmann, Kevin Gómez, and Erhard Rahm. Analyzing extended property graphs with apache flink. In *Proceedings of the 1st ACM SIGMOD Workshop on Network Data Analytics*, pages 1–8, 2016.
- [22] Vasiliki Kalavri, John Liagouris, Moritz Hoffmann, Desislava Dimitrova, Matthew Forshaw, and Timothy Roscoe. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 783–798, Carlsbad, CA, October 2018. USENIX Association.
- [23] R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1):35–45, 03 1960.
- [24] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S Gunawi, Cody Hammock, et al. Lessons learned from the chameleon testbed. In *2020 USENIX annual technical conference (USENIX ATC 20)*, pages 219–233, 2020.
- [25] Abeer Khaleq and Ilkyeun Ra. Intelligent microservices autoscaling module using reinforcement learning. *Cluster Computing*, 26:1–12, 04 2023.
- [26] Romeo Kienzler, Rémy Bruggmann, Anand Ranganathan, and Nesime Tatbul. Large-scale dna sequence analysis in the cloud: a stream-based approach. In *Proceedings of the 2011 International Conference on Parallel Processing - Volume 2*, Euro-Par’11, pages 467–476, Berlin, Heidelberg, 2011. Springer-Verlag.

- [27] Alok Gautam Kumbhare, Yogesh Simmhan, Marc Frincu, and Viktor K. Prasanna. Reactive resource provisioning heuristics for dynamic dataflows on cloud infrastructure. *IEEE Transactions on Cloud Computing*, 3(2):105–118, 2015.
- [28] Shiyuan Liu, Yunchun Li, Hailong Yang, Ming Dun, Chen Chen, Huaitao Zhang, and Wei Li. QAAS: quick accurate auto-scaling for streaming processing. *Frontiers of Computer Science*, 18(1):181–201, 08 2023.
- [29] Ovidiu-Cristian Marcu and Pascal Bouvry. Big data stream processing. Technical report, University of Luxembourg, September 2024.
- [30] Benjamin J. J. Pfister, Dominik Scheinert, Morgan K. Geldenhuys, and Odej Kao. Daedalus: Self-adaptive horizontal autoscaling for resource efficiency of distributed stream processing systems, 2024.
- [31] Henriette Röger and Ruben Mayer. A comprehensive survey on parallelization and elasticity in stream processing. *ACM Comput. Surv.*, 52(2), April 2019.
- [32] Nilabja Roy, Abhishek Dubey, and Aniruddha Gokhale. Efficient autoscaling in the cloud using predictive models for workload forecasting. In *2011 IEEE 4th International Conference on Cloud Computing*, pages 500–507. IEEE, 2011.
- [33] Olubisi Runsewe and Nancy Samaan. Cloud resource scaling for big data streaming applications using a layered multi-dimensional hidden markov model. In *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 848–857, 2017.
- [34] Anshu Shukla, Shilpa Chaturvedi, and Yogesh Simmhan. Riotbench: A real-time iot benchmark for distributed stream processing platforms. *Concurrency and Computation: Practice and Experience*, 29(21), 2017.
- [35] Maninder Pal Singh, Mohammad A. Hoque, and Sasu Tarkoma. A survey of systems for massive stream analytics, 2016.
- [36] Rodrigo Araujo Lima Torres and Marcelo Ladeira. A proposal for online analysis and identification of fraudulent financial transactions. In *2020*

- 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 240–245, 2020.
- [37] Giselle Van Dongen and Dirk Van den Poel. Evaluation of stream processing frameworks. *IEEE Transactions on Parallel and Distributed Systems*, 31(8):1845–1858, 2020.
- [38] Balázs Varga, Márton Balassi, and Attila Kiss. Towards autoscaling of apache flink jobs. *Acta Universitatis Sapientiae. Informatica*, 13(1):39–59, 2021.
- [39] Mark J Willis. Proportional-integral-derivative control. *Dept. of Chemical and Process Engineering University of Newcastle*, 6, 1999.
- [40] Li Yang and Abdallah Shami. A multi-stage automated online network data stream analytics framework for iiot systems. *IEEE Transactions on Industrial Informatics*, 19(2):2107–2116, 2023.
- [41] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: cluster computing with working sets. In *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*, HotCloud’10, page 10, USA, 2010. USENIX Association.